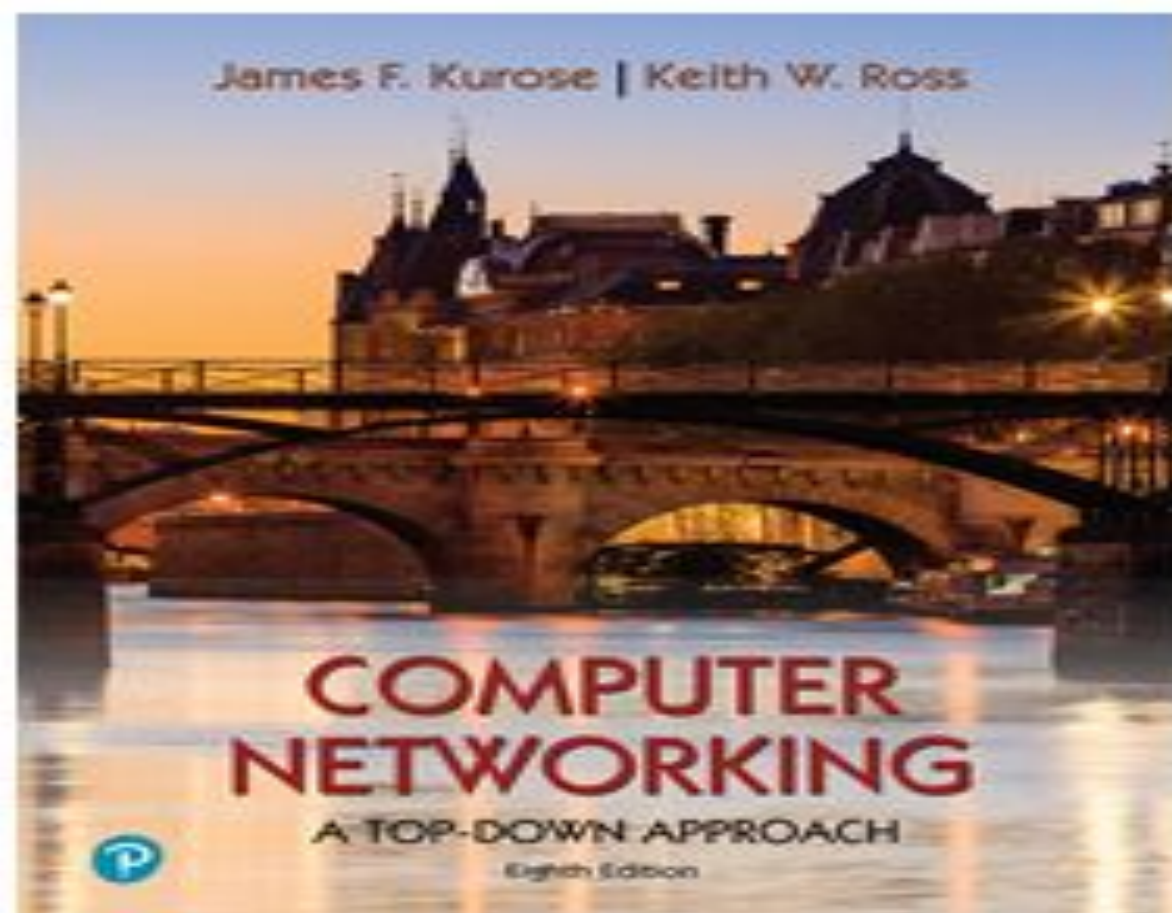




Computer Networking: A Top-Down Approach

8th edition

Jim Kurose, Keith Ross

Pearson, 2020

Chapter 1 Computer Networks and the Internet

1.1 What Is the Internet?

In this book, we'll use the public Internet, a specific computer network, as our principal vehicle for discussing computer networks and their protocols.

But what *is* the Internet? There are a couple of ways to answer this question.

First, we can describe the nuts and bolts of the Internet, that is, the basic hardware and software components that make up the Internet.

Second, we can describe the Internet in terms of a networking infrastructure that provides services to distributed applications.

1.1.1 A Nuts-and-Bolts Description

The Internet is a computer network that interconnects billions of computing devices throughout the world.

Not too long ago, these computing devices were primarily traditional desktop computers, Linux workstations, and so-called servers that store and transmit information such as Web pages and e-mail messages.

Increasingly, however, users connect to the Internet with smartphones and tablets—today, close to half of the world’s population are active mobile Internet users.

Furthermore, non-traditional Internet “things” such as TVs, gaming consoles, thermostats, home security systems, home appliances, watches, eye glasses, cars, traffic control systems, and more are being connected to the Internet.

In Internet jargon, all of these devices are called **hosts** or **end systems**.

By some estimates, there were about 18 billion devices connected to the Internet in 2017, and the number will reach 28.5 billion by 2022 [Cisco VNI 2020].

End systems are connected together by a network of **communication links** and **packet switches**.

Different links can transmit data at different rates, with the **transmission rate** of a link measured in bits/second.

When one end system has data to send to another end system, the sending end system segments the data and adds header bytes to each segment.

The resulting packages of information, known as **packets** in the jargon of computer networks, are then sent through the network to the destination end system, where they are reassembled into the original data.

A packet switch takes a packet arriving on one of its incoming communication links and forwards that packet on one of its outgoing communication links.

Packet switches come in many shapes and flavors, but the two most prominent types in today's Internet are **routers** and **link-layer switches**.

Both types of switches forward packets toward their ultimate destinations.

Link-layer switches are typically used in access networks, while routers are typically used in the network core.

The sequence of communication links and packet switches traversed by a packet from the sending end system to the receiving end system is known as a **route** or **path** through the network.

End systems access the Internet through **Internet Service Providers (ISPs)**, including residential ISPs such as local cable or telephone companies; corporate ISPs; university ISPs; ISPs that provide WiFi access in airports, hotels, coffee shops, and other public places; and cellular data ISPs, providing mobile access to our smartphones and other devices.

Each ISP is in itself a network of packet switches and communication links. ISPs also provide Internet access to content providers, connecting servers directly to the Internet.

The Internet is all about connecting end systems to each other, so the ISPs that provide access to end systems must also be interconnected.

These lower-tier ISPs are thus interconnected through national and international upper-tier ISPs and these upper-tier ISPs are connected directly to each other.

An upper-tier ISP consists of high-speed routers interconnected with high-speed fiber-optic links.

Each ISP network, whether upper-tier or lower-tier, is managed independently,

End systems, packet switches, and other pieces of the Internet run **protocols** that control the sending and receiving of information within the Internet.

The **Transmission Control Protocol (TCP)** and the **Internet Protocol (IP)** are two of the most important protocols in the Internet.

The Internet's principal protocols are collectively known as **TCP/IP**.

Internet standards are developed by the Internet Engineering Task Force (IETF) [IETF 2020].

The IETF standards documents are called **requests for comments (RFCs)**.

RFCs started out as general requests for comments to resolve network and protocol design problems that faced the precursor to the Internet [Allman 2011].

RFCs tend to be quite technical and detailed.

They define protocols such as TCP, IP, HTTP (for the Web), and SMTP (for e-mail).

Other bodies also specify standards for network components, most notably for network links. The IEEE 802 LAN Standards Committee [IEEE 802 2020], for example, specifies the Ethernet and wireless WiFi standards.

1.1.2 A Services Description

But we can also describe the Internet from an entirely different angle—namely, as *an infrastructure that provides services to applications*.

In addition to traditional applications such as e-mail and Web surfing, Internet applications include mobile smartphone and tablet applications, including Internet messaging, mapping with real-time road-traffic information, music streaming movie and television streaming, online social media, video conferencing, multi-person games, and location-based recommendation systems.

The applications are said to be **distributed applications**, since they involve multiple end systems that exchange data with each other.

Importantly, Internet applications run on end systems—they do not run in the packet switches in the network core.

Because applications run on end systems, you are going to need to write programs that run on the end systems.

You might, for example, write your programs in Java, C, or Python.

Now, because you are developing a distributed Internet application, the programs running on the different end systems will need to send data to each other.

And here we get to a central issue—one that leads to the alternative way of describing the Internet as a platform for applications.

How does one program running on one end system instruct the Internet to deliver data to another program running on another end system?

End systems attached to the Internet provide a **socket interface** that specifies how a program running on one end system asks the Internet infrastructure to deliver data to a specific destination program running on another end system.

This Internet socket interface is a set of rules that the sending program must follow so that the Internet can deliver the data to the destination program.

the Internet provides multiple services to its applications.

When you develop an Internet application, you too must choose one of the Internet's services for your application.

1.1.3 What Is a Protocol?

A Human Analogy

Consider what you do when you want to ask someone for the time of day.

Human protocol (or good manners, at least) dictates that one first offer a greeting (the first) to initiate communication with someone else.

The typical response to a “Hi” is a returned “Hi” message.

Implicitly, one then takes a cordial “Hi” response as an indication that one can proceed and ask for the time of day.

A different response to the initial “Hi” (such as “Don’t bother me!” or “I don’t speak English,” or some unprintable reply) might indicate an unwillingness or inability to communicate.

In this case, the human protocol would be not to ask for the time of day.

Sometimes one gets no response at all to a question, in which case one typically gives up asking that person for the time.

Note that in our human protocol, *there are specific messages we send, and specific actions we take in response to the received reply messages or other events* (such as no reply within some given amount of time).

Clearly, transmitted and received messages, and actions taken when these messages are sent or received or other events occur, play a central role in a human protocol.

Network Protocols

A network protocol is similar to a human protocol, except that the entities exchanging messages and taking actions are hardware or software components of some device (for example, computer, smartphone, tablet, router, or other network-capable device).

All activity in the Internet that involves two or more communicating remote entities is governed by a protocol.

Protocols are running everywhere in the Internet, and consequently much of this book is about computer network protocols.

As an example of a computer network protocol with which you are probably familiar, consider what happens when you make a request to a Web server, that is, when you type the URL of a Web page into your Web browser.

First, your computer will send a connection request message to the Web server and wait for a reply. The Web server will eventually receive your connection request message and return a connection reply message.

Knowing that it is now OK to request the Web document, your computer then sends the name of the Web page it wants to fetch from that Web server in a GET message.

Finally, the Web server returns the Web page (file) to your computer.

*A **protocol** defines the format and the order of messages exchanged between two or more communicating entities, as well as the actions taken on the transmission and/or receipt of a message or other event.*

1.2 The Network Edge

We begin in this section at the edge of the network and look at the components with which we are most familiar—namely, the computers, smartphones and other devices that we use on a daily basis.

Recall from the previous section that in computer networking jargon, the computers and other devices connected to the Internet are often referred to as end systems.

They are referred to as end systems because they sit at the edge of the Internet.

The Internet's end systems include desktop computers (e.g., desktop PCs, Macs, and Linux boxes), servers (e.g., Web and e-mail servers), and mobile devices (e.g., laptops, smartphones, and tablets).

End systems are also referred to as *hosts* because they host (that is, run) application programs such as a Web browser program, a Web server program, an e-mail client program, or an e-mail server program.

Throughout this book we will use the terms hosts and end systems interchangeably; that is, *host = end system*.

Hosts are sometimes further divided into two categories: **clients** and **servers**.

Informally, clients tend to be desktops, laptops, smartphones, and so on, whereas servers tend to be more powerful machines that store and distribute Web pages, stream video, relay e-mail, and so on.

Today, most of the servers from which we receive search results, e-mail, Web pages, videos and mobile app content reside in large **data centers**.

1.2.1 Access Networks

let's next consider the access network—the network that physically connects an end system to the first router (also known as the “edge router”) on a path from the end system to any other distant end system.

Home Access: DSL, Cable, FTTH, and 5G Fixed Wireless

let's begin our overview of access networks by considering how homes connect to the Internet.

Today, the two most prevalent types of broadband residential access are **digital subscriber line (DSL)** and cable.

A residence typically obtains DSL Internet access from the same local telephone company (telco) that provides its wired local phone access.

Thus, when DSL is used, a customer's telco is also its ISP.

each customer's DSL modem uses the existing telephone line exchange data with a digital subscriber line access multiplexer (DSLAM) located in the telco's local central office (CO).

The home's DSL modem takes digital data and translates it to high- frequency tones for transmission over telephone wires to the CO; the analog signals from many such houses are translated back into digital format at the DSLAM.

The residential telephone line carries both data and traditional telephone signals simultaneously, which are encoded at different frequencies:

On the customer side, a splitter separates the data and telephone signals arriving to the home and forwards the data signal to the DSL modem.

On the telco side, in the CO, the DSLAM separates the data and phone signals and sends the data into the Internet.

Hundreds or even thousands of households connect to a single DSLAM.

Engineers have expressly designed DSL for short distances between the home and the CO; generally, if the residence is not located within 5 to 10 miles of the CO, the residence must resort to an alternative form of Internet access.

While DSL makes use of the telco's existing local telephone infrastructure, **cable Internet access** makes use of the cable television company's existing cable television infrastructure. A residence obtains cable Internet access from the same company that provides its cable television.

As illustrated in Figure 1.6, fiber optics connect the cable head end to neighborhood-level junctions, from which traditional coaxial cable is then used to reach individual houses and apartments.

Each neighborhood junction typically supports 500 to 5,000 homes.

Because both fiber and coaxial cable are employed in this system, it is often referred to as hybrid fiber coax (HFC).

Cable internet access requires special modems, called cable modems.

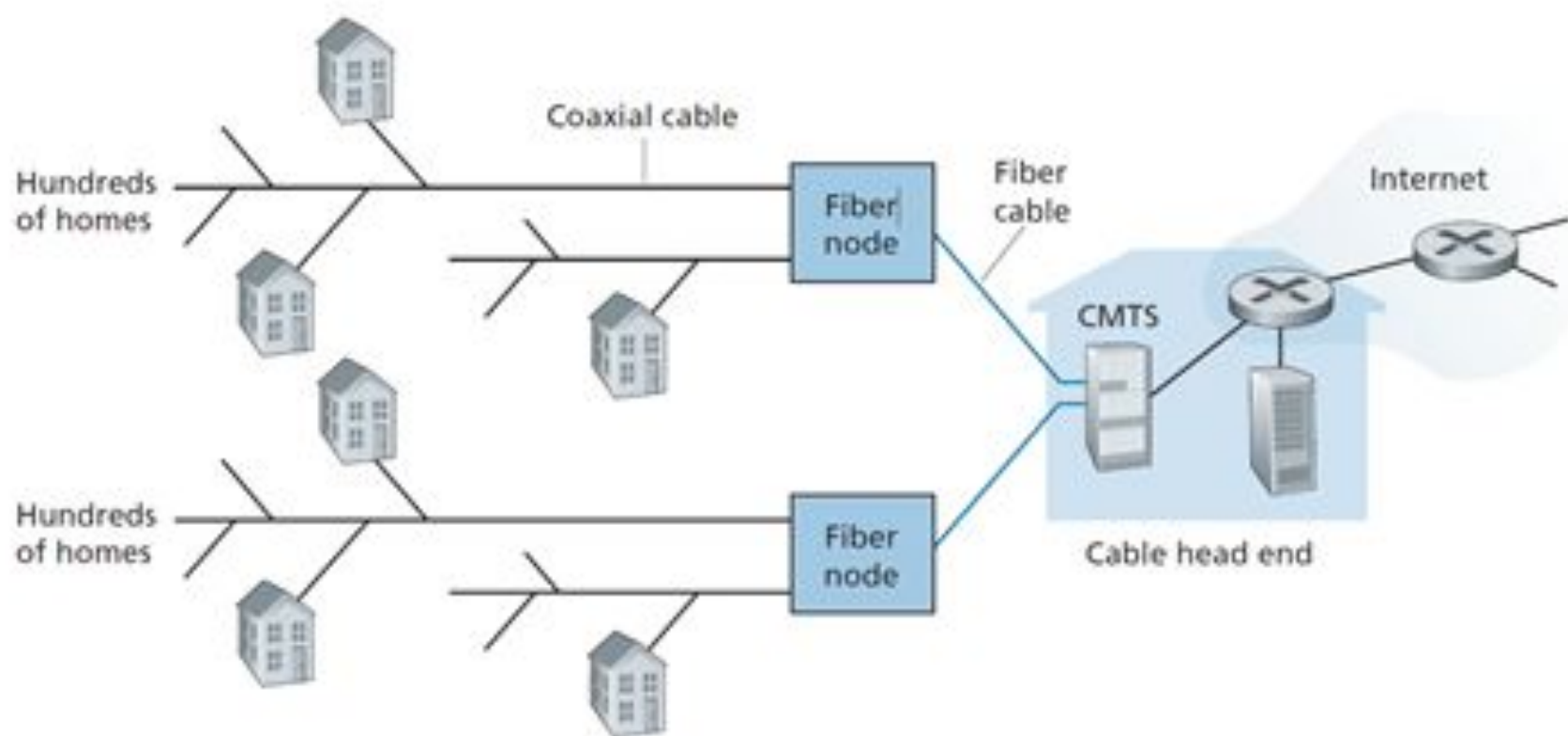


Figure 1.6 • A hybrid fiber-coaxial access network

As with a DSL modem, the cable modem is typically an external device and connects to the home PC through an Ethernet port.

At the cable head end, the cable modem termination system (CMTS) serves a similar function as the DSL network's DSLAM—turning the analog signal sent from the cable modems in many downstream homes back into digital format.

Cable modems divide the HFC network into two channels, a downstream and an upstream channel.

As with DSL, access is typically asymmetric, with the downstream channel typically allocated a higher transmission rate than the upstream channel.

One important characteristic of cable Internet access is that it is a shared broadcast medium.

In particular, every packet sent by the head end travels downstream on every link to every home and every packet sent by a home travels on the upstream channel to the head end.

For this reason, if several users are simultaneously downloading a video file on the downstream channel, the actual rate at which each user receives its video file will be significantly lower than the aggregate cable downstream rate.

On the other hand, if there are only a few active users and they are all Web surfing, then each of the users may actually receive Web pages at the full cable downstream rate, because the users will rarely request a Web page at exactly the same time.

Although DSL and cable networks currently represent the majority of residential broadband access in the United States, an up-and-coming technology that provides even higher speeds is **fiber to the home (FTTH)** [Fiber Broadband 2020].

As the name suggests, the FTTH concept is simple—provide an optical fiber path from the CO directly to the home.

FTTH can potentially provide Internet access rates in the gigabits per second range.

There are several competing technologies for optical distribution from the CO to the homes.

The simplest optical distribution network is called direct fiber,

In addition to DSL, Cable, and FTTH, **5G fixed wireless** is beginning to be deployed.

Access in the Enterprise (and the Home): Ethernet and WiFi

On corporate and university campuses, and increasingly in home settings, a local area network (LAN) is used to connect an end system to the edge router.

Although there are many types of LAN technologies, Ethernet is by far the most prevalent access technology in corporate, university, and home networks.

Ethernet users use twisted-pair copper wire to connect to an Ethernet switch,

The Ethernet switch, or a network of such interconnected switches, is then in turn connected into the larger Internet.

Increasingly, however, people are accessing the Internet wirelessly from laptops, smartphones, tablets, and other “things”.

In a wireless LAN setting, wireless users transmit/receive packets to/from an access point that is connected into the enterprise’s network

A wireless LAN user must typically be within a few tens of meters of the access point.

Wireless LAN access based on IEEE 802.11 technology, more colloquially known as WiFi, is now just about everywhere—universities, business offices, cafes, airports, homes, and even in airplanes.

Even though Ethernet and WiFi access networks were initially deployed in enterprise (corporate, university) settings, they are also common components of home networks.

Many homes combine broadband residential access (that is, cable modems or DSL) with these inexpensive wireless LAN technologies to create powerful home networks

Wide-Area Wireless Access: 3G and LTE 4G and 5G

Mobile devices such as iPhones and Android devices are being used to message, share photos in social networks, make mobile payments, watch movies, stream music, and much more while on the run.

These devices employ the same wireless infrastructure used for cellular telephony to send/receive packets through a base station that is operated by the cellular network provider.

Unlike WiFi, a user need only be within a few tens of kilometers (as opposed to a few tens of meters) of the base station.

Telecommunications companies have made enormous investments in so-called fourth-generation (4G) wireless, which provides real-world download speeds of up to 60 Mbps.

But even higher-speed wide-area access technologies—a fifth-generation (5G) of wide-area wireless networks—are already being deployed.

1.2.2 Physical Media

For example, we said that HFC uses a combination of fiber cable and coaxial cable.

We said that DSL and Ethernet use copper wire.

And we said that mobile access networks use the radio spectrum.

In this subsection, we provide a brief overview of these and other transmission media that are commonly used in the Internet.

In order to define what is meant by a physical medium, let us reflect on the brief life of a bit. Consider a bit traveling from one end system, through a series of links and routers, to another end system.

This poor bit gets kicked around and transmitted many, many times! The source end system first transmits the bit, and shortly thereafter the first router in the series receives the bit; the first router then transmits the bit, and shortly thereafter the second router receives the bit; and so on.

Thus our bit, when traveling from source to destination, passes through a series of transmitter-receiver pairs.

Thus our bit, when traveling from source to destination, passes through a series of transmitter-receiver pairs.

For each transmitter-receiver pair, the bit is sent by propagating electromagnetic waves or optical pulses across a **physical medium**.

The physical medium can take many shapes and forms and does not have to be of the same type for each transmitter-receiver pair along the path.

Examples of physical media include twisted-pair copper wire, coaxial cable, multimode fiber-optic cable, terrestrial radio spectrum, and satellite radio spectrum.

Physical media fall into two categories: **guided media** and **unguided media**.

With guided media, the waves are guided along a solid medium, such as a fiber-optic cable, a twisted-pair copper wire, or a coaxial cable.

With unguided media, the waves propagate in the atmosphere and in outer space, such as in a wireless LAN or a digital satellite channel.

But before we get into the characteristics of the various media types, let us say a few words about their costs.

The actual cost of the physical link (copper wire, fiberoptic cable, and so on) is often relatively minor compared with other networking costs.

For this reason, many builders install twisted pair, optical fiber, and coaxial cable in every room in a building.

Twisted-Pair Copper Wire

The least expensive and most commonly used guided transmission medium is twisted-pair copper wire.

For over a hundred years it has been used by telephone networks.

In fact, more than 99 percent of the wired connections from the telephone handset to the local telephone switch use twisted-pair copper wire.

Twisted pair consists of two insulated copper wires, each about 1 mm thick, arranged in a regular spiral pattern.

Unshielded twisted pair (UTP) is commonly used for computer networks within a building, that is, for LANs.

Data rates for LANs using twisted pair today range from 10 Mbps to 10 Gbps.

The data rates that can be achieved depend on the thickness of the wire and the distance between transmitter and receiver.

When fiber-optic technology emerged in the 1980s, many people disparaged twisted pair because of its relatively low bit rates.

Some people even felt that fiber-optic technology would completely replace twisted pair. But twisted pair did not give up so easily.

Modern twisted-pair technology, such as category 6a cable, can achieve data rates of 10 Gbps for distances up to a hundred meters.

In the end, twisted pair has emerged as the dominant solution for high-speed LAN networking.

As discussed earlier, twisted pair is also commonly used for residential Internet access.

We also saw that DSL (digital subscriber line) technology has enabled residential users to access the Internet at tens of Mbps over twisted pair (when users live close to the ISP's central office).

Coaxial Cable

Like twisted pair, coaxial cable consists of two copper conductors, but the two conductors are concentric rather than parallel.

With this construction and special insulation and shielding, coaxial cable can achieve high data transmission rates.

Coaxial cable is quite common in cable television systems.

As we saw earlier, cable television systems have recently been coupled with cable modems to provide residential users with Internet access at rates of hundreds of Mbps.

In cable television and cable Internet access, the transmitter shifts the digital signal to a specific frequency band, and the resulting analog signal is sent from the transmitter to one or more receivers.

Coaxial cable can be used as a guided **shared medium**.

Specifically, a number of end systems can be connected directly to the cable, with each of the end systems receiving whatever is sent by the other end systems.

Fiber Optics

An optical fiber is a thin, flexible medium that conducts pulses of light, with each pulse representing a bit.

A single optical fiber can support tremendous bit rates, up to tens or even hundreds of gigabits per second.

They are immune to electromagnetic interference, have very low signal attenuation up to 100 kilometers, and are very hard to tap.

These characteristics have made fiber optics the preferred long-haul guided transmission media, particularly for overseas links.

Many of the long-distance telephone networks in the United States and elsewhere now use fiber optics exclusively.

Fiber optics is also prevalent in the backbone of the Internet.

However, the high cost of optical devices—such as transmitters, receivers, and switches—has hindered their deployment for short-haul transport, such as in a LAN or into the home in a residential access network.

Terrestrial Radio Channels

Radio channels carry signals in the electromagnetic spectrum.

They are an attractive medium because they require no physical wire to be installed, can penetrate walls, provide connectivity to a mobile user, and can potentially carry a signal for long distances.

The characteristics of a radio channel depend significantly on the propagation environment and the distance over which a signal is to be carried.

Terrestrial radio channels can be broadly classified into three groups:

- those that operate over very short distance (e.g., with one or two meters);

- those that operate in local area s, typically spanning from ten to a few hundred meters;

- and those that operate in the wide area, spanning tens of kilometers.

Personal devices such as wireless headsets, keyboards, and medical devices operate over short distances;

the wireless LAN technologies use local-area radio channels;

the cellular access technologies use wide-area radio channels.

Satellite Radio Channels

A communication satellite links two or more Earth-based microwave transmitter/receivers, known as ground stations.

The satellite receives transmissions on one frequency band, regenerates the signal using a repeater and transmits the signal on another frequency.

Two types of satellites are used in communications: **geostationary satellites** and **low-earth orbiting (LEO) satellites**.

Geostationary satellites permanently remain above the same spot on Earth.

This stationary presence is achieved by placing the satellite in orbit at 36,000 kilometers above Earth's surface.

This huge distance from ground station through satellite back to ground station introduces a substantial signal propagation delay of 280 milliseconds.

Nevertheless, satellite links, which can operate at speeds of hundreds of Mbps, are often used in areas without access to DSL or cable-based Internet access.

LEO satellites are placed much closer to Earth and do not remain permanently above one spot on Earth. They rotate around Earth (just as the Moon does) and may communicate with each other, as well as with ground stations.

1.3 The Network Core

let us now delve more deeply inside the network core—the mesh of packet switches and links that interconnects the Internet's end systems.

1.3.1 Packet Switching

In a network application, end systems exchange **messages** with each other. Messages can contain anything the application designer wants.

To send a message from a source end system to a destination end system, the source breaks long messages into smaller chunks of data known as **packets**.

Between source and destination, each packet travels through communication links and **packet switches** (for which there are two predominant types, **routers** and **link-layer switches**).

link with transmission rate R bits/sec

Store-and-Forward Transmission

Most packet switches use **store-and-forward transmission** at the inputs to the links.

Store-and-forward transmission means that the packet switch must receive the entire packet before it can begin to transmit the first bit of the packet onto the outbound link.

To explore store-and-forward transmission in more detail, consider a simple network consisting of two end systems connected by a single router, as shown in Figure 1.11.

in this simple example, the router has the rather simple task of transferring a packet from one (input) link to the only other attached link.

In this example, the source has three packets, each consisting of L bits, to send to the destination.

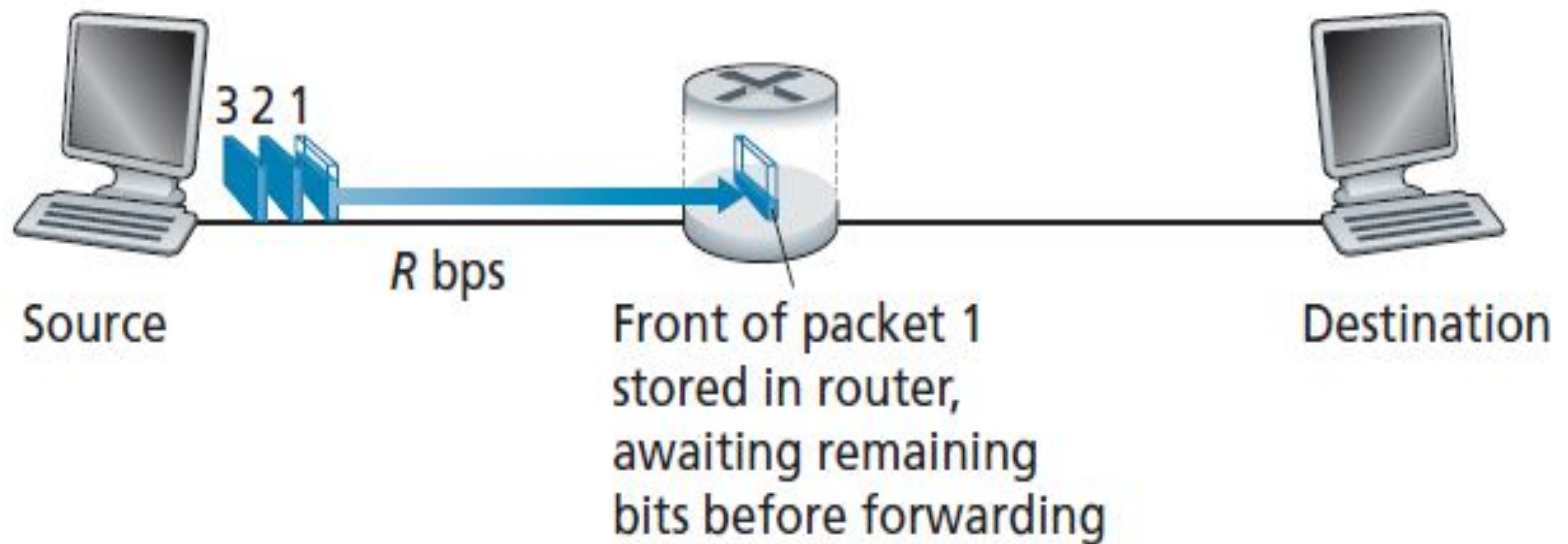


Figure 1.11 ♦ Store-and-forward packet switching

At the snapshot of time shown in Figure 1.11, the source has transmitted some of packet 1, and the front of packet 1 has already arrived at the router.

Because the router employs store-and-forwarding, at this instant of time, the router cannot transmit the bits it has received; instead it must first buffer (i.e., “store”) the packet’s bits. Only after the router has received *all* of the packet’s bits can it begin to transmit (i.e., “forward”) the packet onto the outbound link.

To gain some insight into store-and-forward transmission, let's now calculate the amount of time that elapses from when the source begins to send the packet until the destination has received the entire packet.

The source begins to transmit at time 0;

at time L/R seconds, the source has transmitted the entire packet, and the entire packet has been received and stored at the router

At time L/R seconds, since the router has just received the entire packet, it can begin to transmit the packet onto the outbound link towards the destination;

at time $2L/R$, the router has transmitted the entire packet, and the entire packet has been received by the destination.

Thus, the total delay is $2L/R$.

Now let's calculate the amount of time that elapses from when the source begins to send the first packet until the destination has received all three packets.

As before, at time L/R , the router begins to forward the first packet.

But also at time L/R the source will begin to send the second packet, since it has just finished sending the entire first packet.

Thus, at time $2L/R$, the destination has received the first packet and the router has received the second packet.

Similarly, at time $3L/R$, the destination has received the first two packets and the router has received the third packet.

Finally, at time $4L/R$ the destination has received all three packets!

Let's now consider the general case of sending one packet from source to destination over a path consisting of N links each of rate R (thus, there are $N-1$ routers between source and destination). Applying the same logic as above, we see that the end-to-end delay is:

$$d_{\text{end-to-end}} = N \frac{L}{R} \quad (1.1)$$

Queuing Delays and Packet Loss

Each packet switch has multiple links attached to it.

For each attached link, the packet switch has an **output buffer** (also called an **output queue**), which stores packets that the router is about to send into that link.

The output buffers play a key role in packet switching.

If an arriving packet needs to be transmitted onto a link but finds the link busy with the transmission of another packet, the arriving packet must wait in the output buffer.

Thus, in addition to the store-and-forward delays, packets suffer output buffer **queuing delays**.

These delays are variable and depend on the level of congestion in the network.

Since the amount of buffer space is finite, an arriving packet may find that the buffer is completely full with other packets waiting for transmission.

In this case, **packet loss** will occur—either the arriving packet or one of the already-queued packets will be dropped.

Figure 1.12 illustrates a simple packet-switched network.

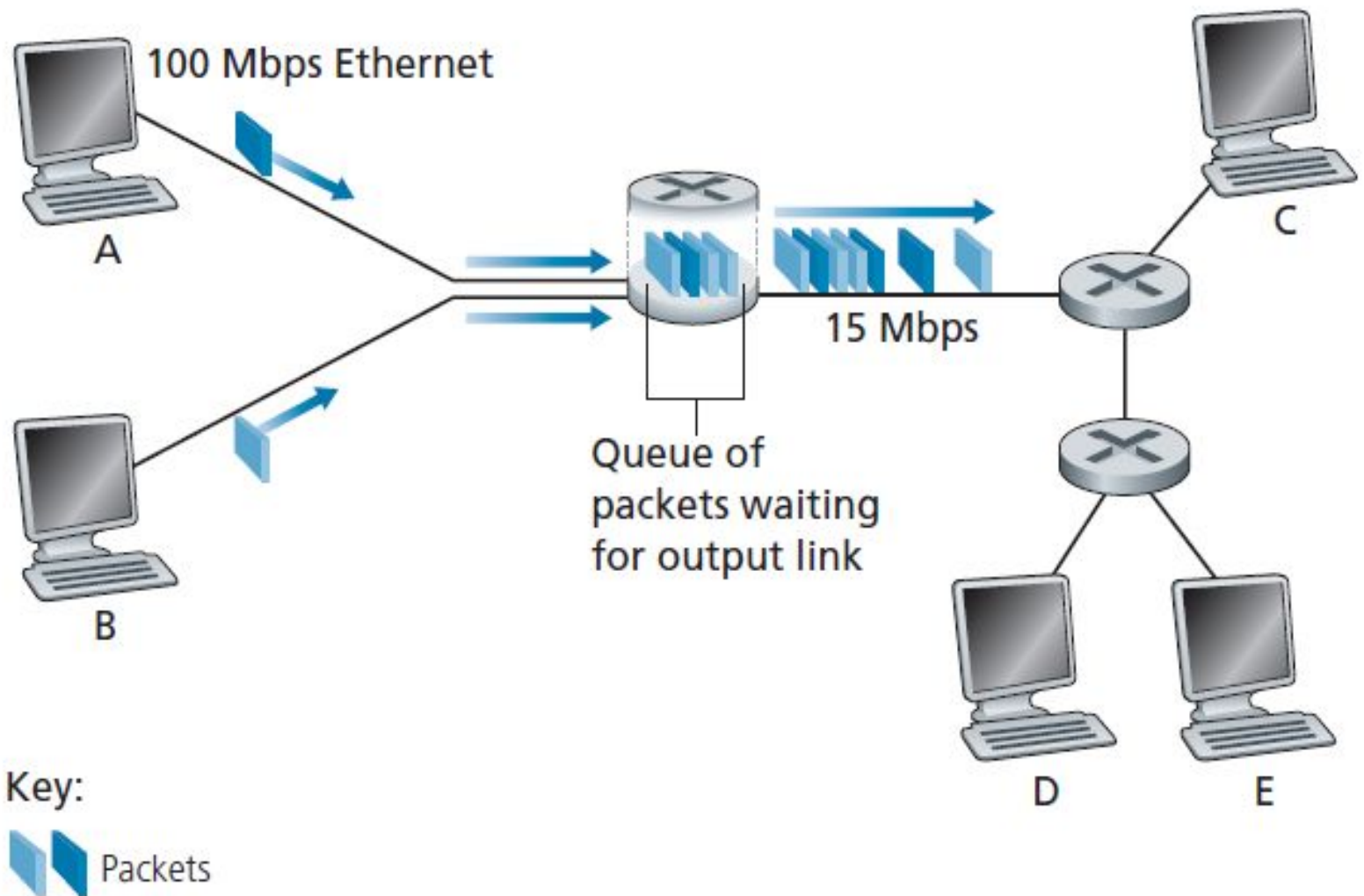


Figure 1.12 ♦ Packet switching

In this figure, all packets have the same length.

Suppose Hosts A and B are sending packets to Host E.

Hosts A and B first send their packets along 100 Mbps Ethernet links to the first router.

The router then directs these packets to the 15 Mbps link.

If, during a short interval of time, the arrival rate of packets to the router exceeds 15 Mbps, congestion will occur at the router as packets queue in the link's output buffer before being transmitted onto the link.

Forwarding Tables and Routing Protocols

Earlier, we said that a router takes a packet arriving on one of its attached communication links and forwards that packet onto another one of its attached communication links.

But how does the router determine which link it should forward the packet onto? Packet forwarding is actually done in different ways in different types of computer networks.

Here, we briefly describe how it is done in the Internet.

In the Internet, every end system has an address called an IP address.

When a source end system wants to send a packet to a destination end system, the source includes the destination's IP address in the packet's header.

When a packet arrives at a router in the network, the router examines a portion of the packet's destination address and forwards the packet to an adjacent router.

More specifically, each router has a **forwarding table** that maps destination addresses to that router's outbound links.

When a packet arrives at a router, the router examines the address and searches its forwarding table, using this destination address, to find the appropriate outbound link. The router then directs the packet to this outbound link.

The end-to-end routing process is analogous to a car driver who does not use maps but instead prefers to ask for directions.

We just learned that a router uses a packet's destination address to index a forwarding table and determine the appropriate outbound link.

we'll note now that the Internet has a number of special **routing protocols** that are used to automatically set the forwarding tables.

1.3.2 Circuit Switching

There are two fundamental approaches to moving data through a network of links and switches: **circuit switching** and **packet switching**.

Having covered packet-switched networks in the previous subsection,

In circuit-switched networks, the resources needed along a path (buffers, link transmission rate) to provide for communication between the end systems are *reserved* for the duration of the communication session between the end systems.

In packet-switched networks, these resources are *not* reserved;

As a simple analogy, consider two restaurants, one that requires reservations and another that neither requires reservations nor accepts them.

For the restaurant that requires reservations, we have to go through the hassle of calling before we leave home.

But when we arrive at the restaurant we can, in principle, immediately be seated and order our meal. For the restaurant that does not require reservations, we don't need to bother to reserve a table.

But when we arrive at the restaurant, we may have to wait for a table before we can be seated.

Traditional telephone networks are examples of circuit-switched networks.

Multiplexing in Circuit-Switched Networks

A circuit in a link is implemented with either **frequency-division multiplexing (FDM)** or **time-division multiplexing (TDM)**.

Packet Switching Versus Circuit Switching

Critics of packet switching have often argued that packet switching is not suitable for real-time services (for example, telephone calls and video conference calls) because of its variable and unpredictable end-to-end delays (due primarily to variable and unpredictable queuing delays).

Proponents of packet switching argue that

- (1) it offers better sharing of transmission capacity than circuit switching and
- (2) it is simpler, more efficient, and less costly to implement than circuit switching.

Generally speaking, people who do not like to hassle with restaurant reservations prefer packet switching to circuit switching.

Although packet switching and circuit switching are both prevalent in today's telecommunication networks, the trend has certainly been in the direction of packet switching.

Even many of today's circuit-switched telephone networks are slowly migrating toward packet switching.

In particular, telephone networks often use packet switching for the expensive overseas portion of a telephone call.

1.3.3 A Network of Networks

We saw earlier that end systems (PCs, smartphones, Web servers, mail servers, and so on) connect into the Internet via an access ISP.

The access ISP can provide either wired or wireless connectivity, using an array of access technologies including DSL, cable, FTTH, Wi-Fi, and cellular.

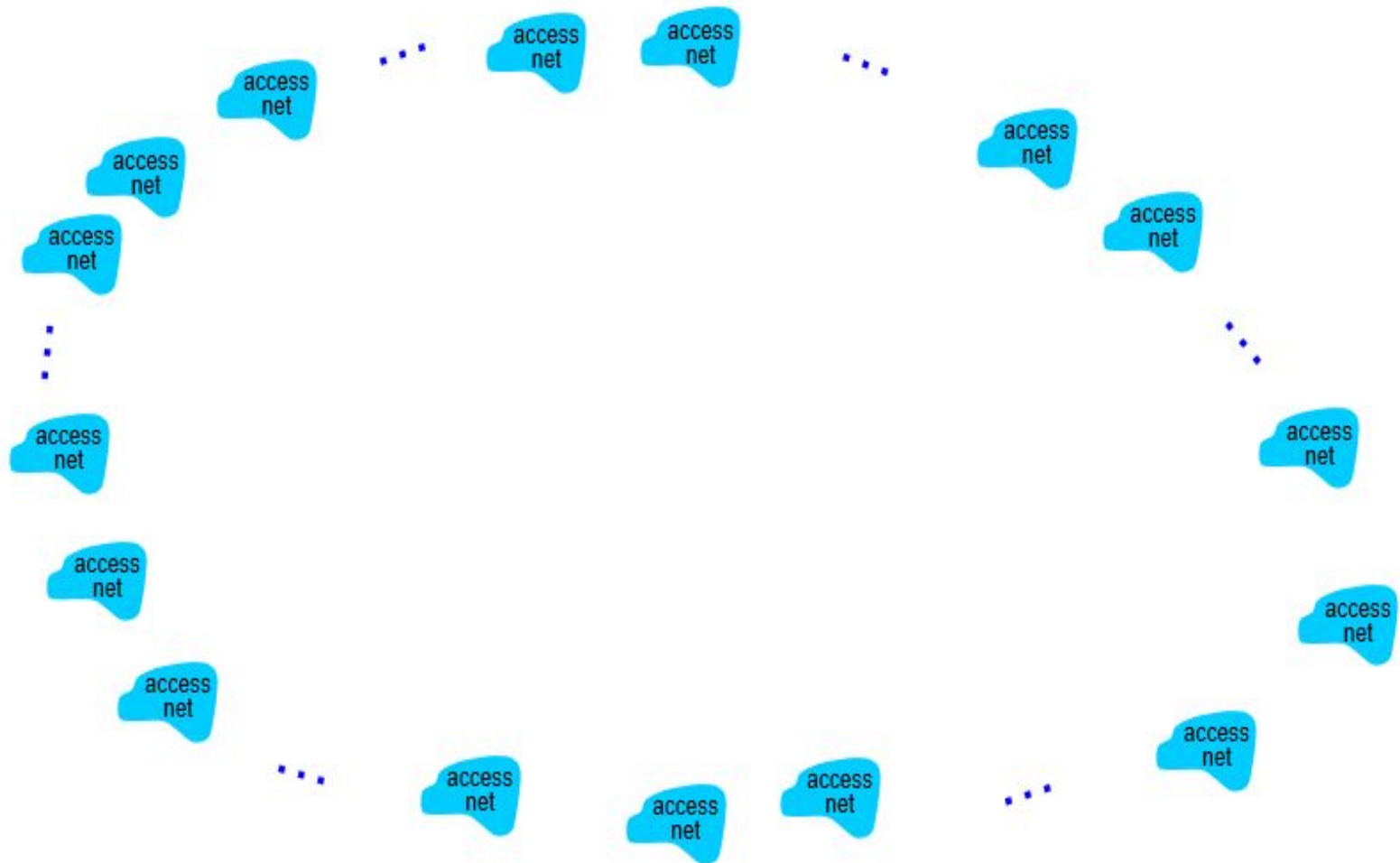
Note that the access ISP does not have to be a telco or a cable company; instead it can be, for example, a university (providing Internet access to students, staff, and faculty), or a company (providing access for its employees).

But connecting end users and content providers into an access ISP is only a small piece of solving the puzzle of connecting the billions of end systems that make up the Internet.

To complete this puzzle, the access ISPs themselves must be interconnected.

This is done by creating a *network of networks*—understanding this phrase is the key to understanding the Internet.

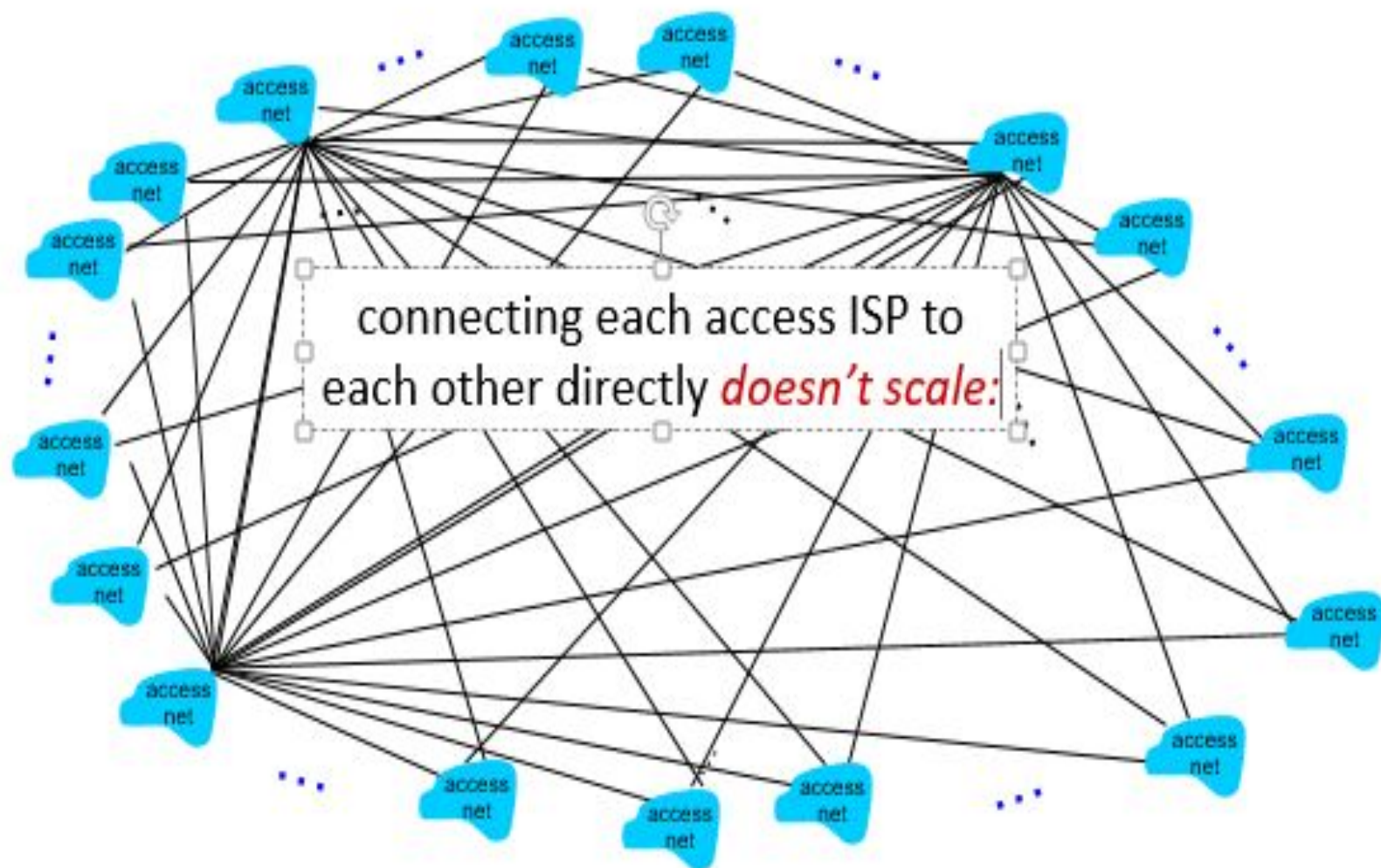
Question: given *millions* of access ISPs, how to connect them together?



In order to understand today's Internet network structure, let's incrementally build a series of network structures, with each new structure being a better approximation of the complex Internet that we have today.

Recall that the overarching goal is to interconnect the access ISPs so that all end systems can send packets to each other.

One naive approach would be to have each access ISP *directly* connect with every other access ISP. Such a mesh design is, of course, much too costly for the access ISPs, as it would require each access ISP to have a separate communication link to each of the hundreds of thousands of other access ISPs all over the world.

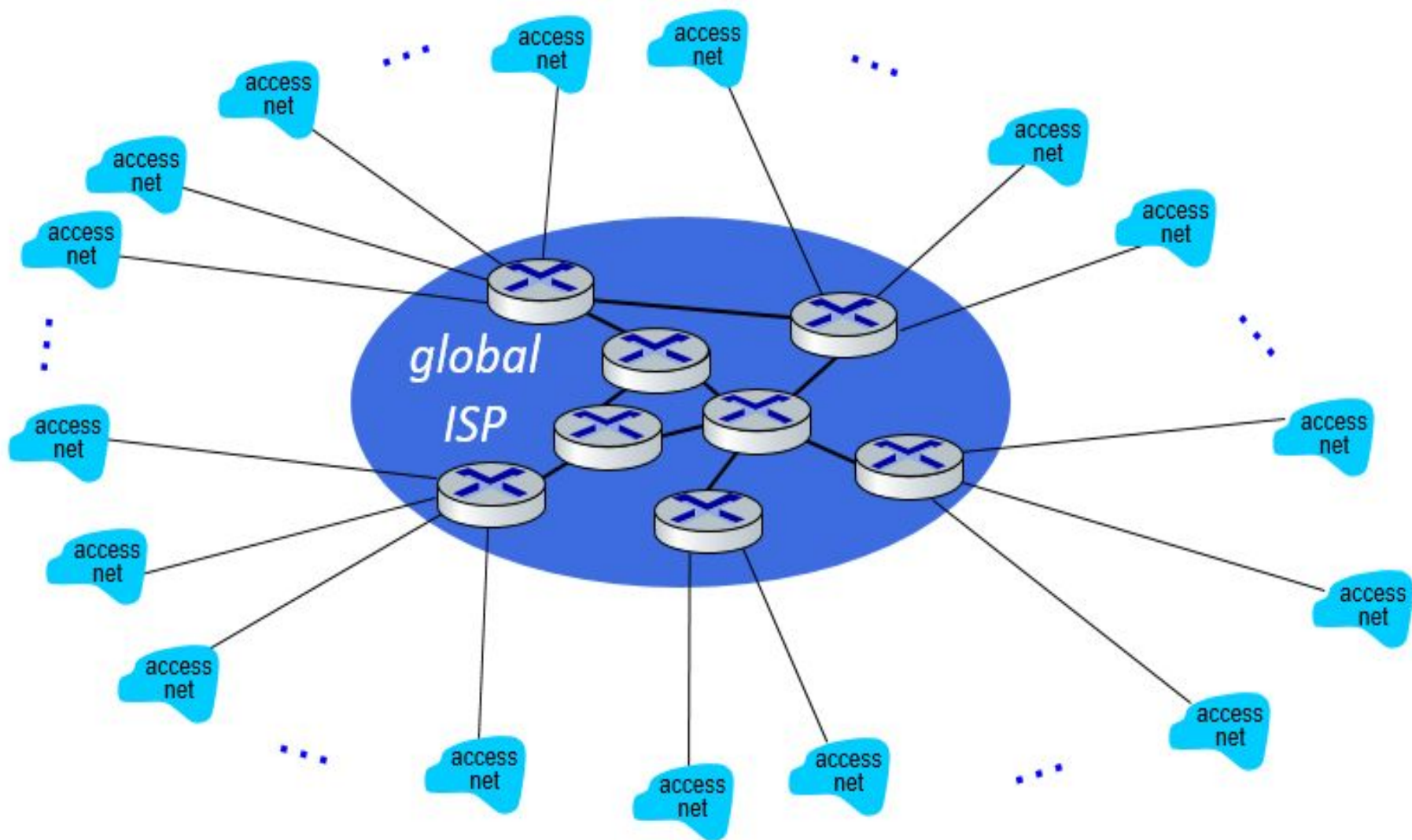


Our first network structure, *Network Structure 1*, interconnects all of the access ISPs with a *single global transit ISP*.

Our (imaginary) global transit ISP is a network of routers and communication links that not only spans the globe, but also has at least one router near each of the hundreds of thousands of access ISPs.

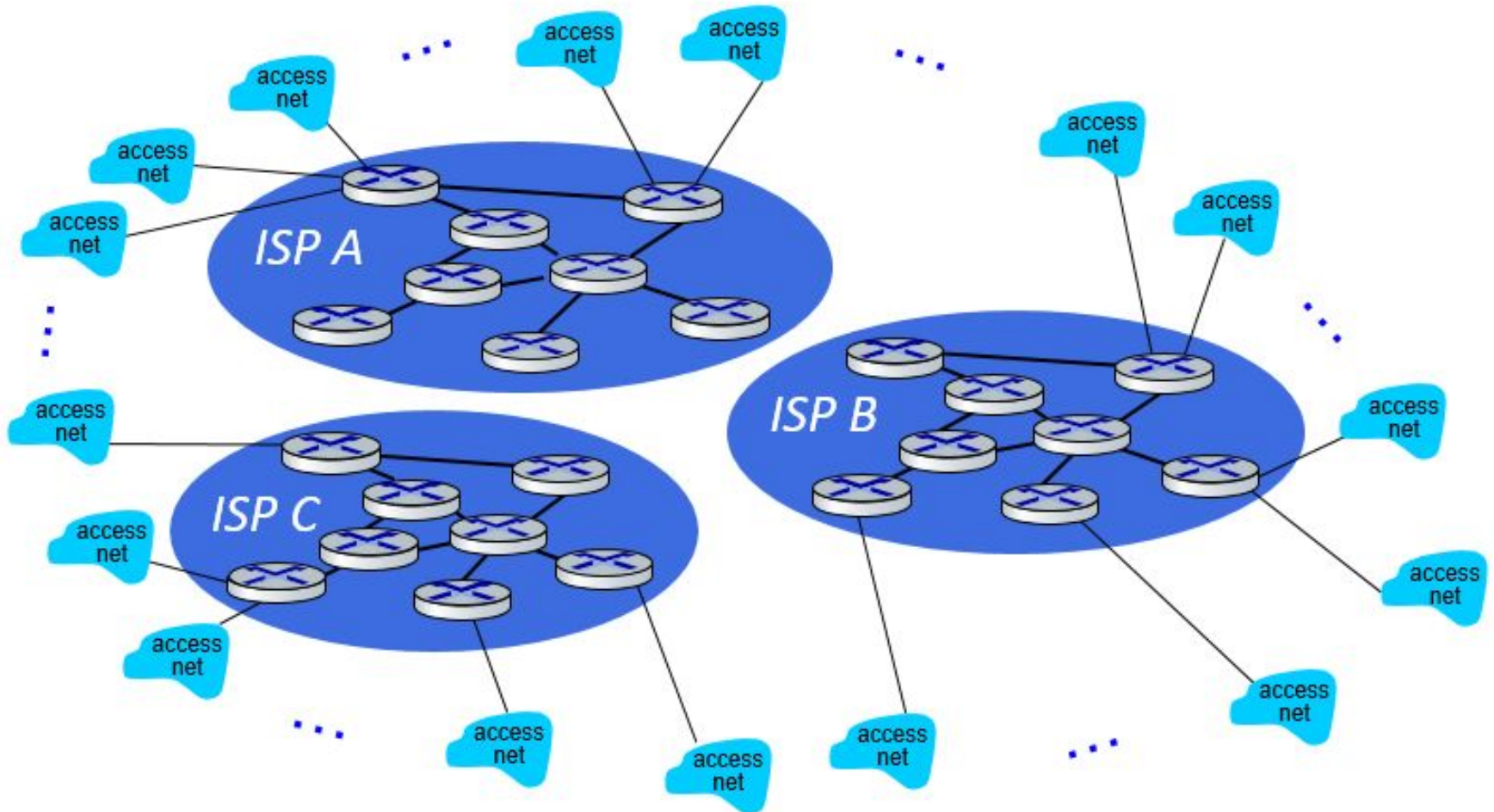
Of course, it would be very costly for the global ISP to build such an extensive network. To be profitable, it would naturally charge each of the access ISPs for connectivity, with the pricing reflecting (but not necessarily directly proportional to) the amount of traffic an access ISP exchanges with the global ISP.

Since the access ISP pays the global transit ISP, the access ISP is said to be a **customer** and the global transit ISP is said to be a **provider**.



Now if some company builds and operates a global transit ISP that is profitable, then it is natural for other companies to build their own global transit ISPs and compete with the original global transit ISP.

This leads to *Network Structure 2*, which consists of the hundreds of thousands of access ISPs and *multiple* global transit ISPs.



The access ISPs certainly prefer Network Structure 2 over Network Structure 1 since they can now choose among the competing global transit providers as a function of their pricing and services.

Note, however, that the global transit ISPs themselves must interconnect: Otherwise access ISPs connected to one of the global transit providers would not be able to communicate with access ISPs connected to the other global transit providers.

Network Structure 2, just described, is a **two-tier hierarchy** with global transit providers residing at the top tier and access ISPs at the bottom tier.

In reality, although some ISPs do have impressive global coverage and do directly connect with many access ISPs, no ISP has presence in each and every city in the world.

Instead, in any given region, there may be a **regional ISP** to which the access ISPs in the region connect.

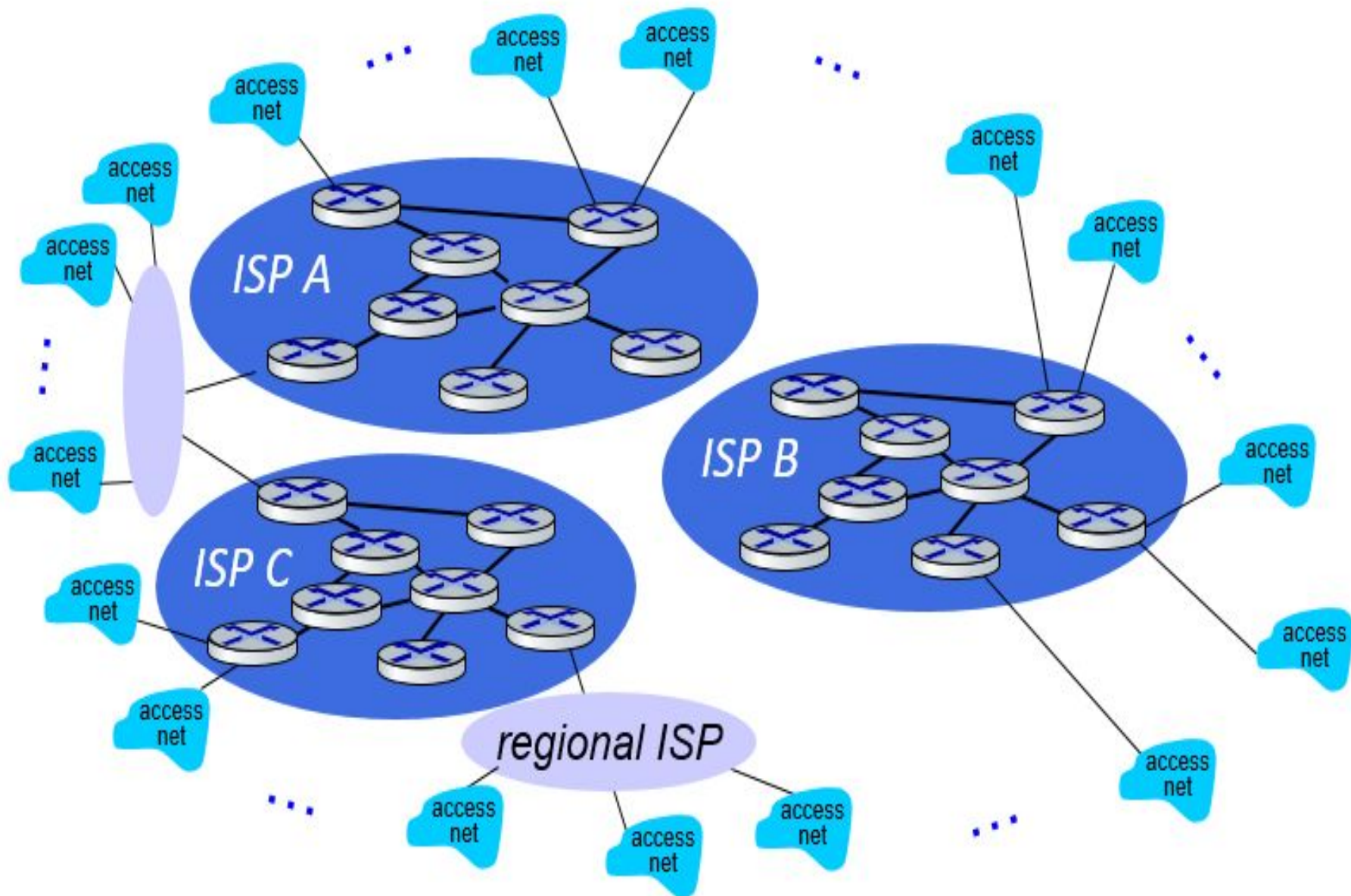
Each regional ISP then connects to **tier-1 ISPs**.

Tier-1 ISPs are similar to our (imaginary) global transit ISP; but tier-1 ISPs, which actually do exist, do not have a presence in every city in the world.

There are approximately a dozen tier-1 ISPs, including Level 3 Communications, AT&T, Sprint, and NTT.

Returning to this network of networks, not only are there multiple competing tier-1 ISPs, there may be multiple competing regional ISPs in a region.

In such a hierarchy, each access ISP pays the regional ISP to which it connects, and each regional ISP pays the tier-1 ISP to which it connects.



(An access ISP can also connect directly to a tier-1 ISP, in which case it pays the tier-1 ISP).

Thus, there is customer-provider relationship at each level of the hierarchy.

Note that the tier-1 ISPs do not pay anyone as they are at the top of the hierarchy.

To further complicate matters, in some regions, there may be a larger regional ISP (possibly spanning an entire country) to which the smaller regional ISPs in that region connect; the larger regional ISP then connects to a tier-1 ISP.

We refer to this **multi-tier hierarchy**, which is still only a crude approximation of today's Internet, as *Network Structure 3*.

To build a network that more closely resembles today's Internet, we must add points of presence (PoPs), multi-homing, peering, and Internet exchange points (IXPs) to the hierarchical Network Structure 3.

A **PoP** is simply a group of one or more routers in the provider's network where customer ISPs can connect into the provider ISP.

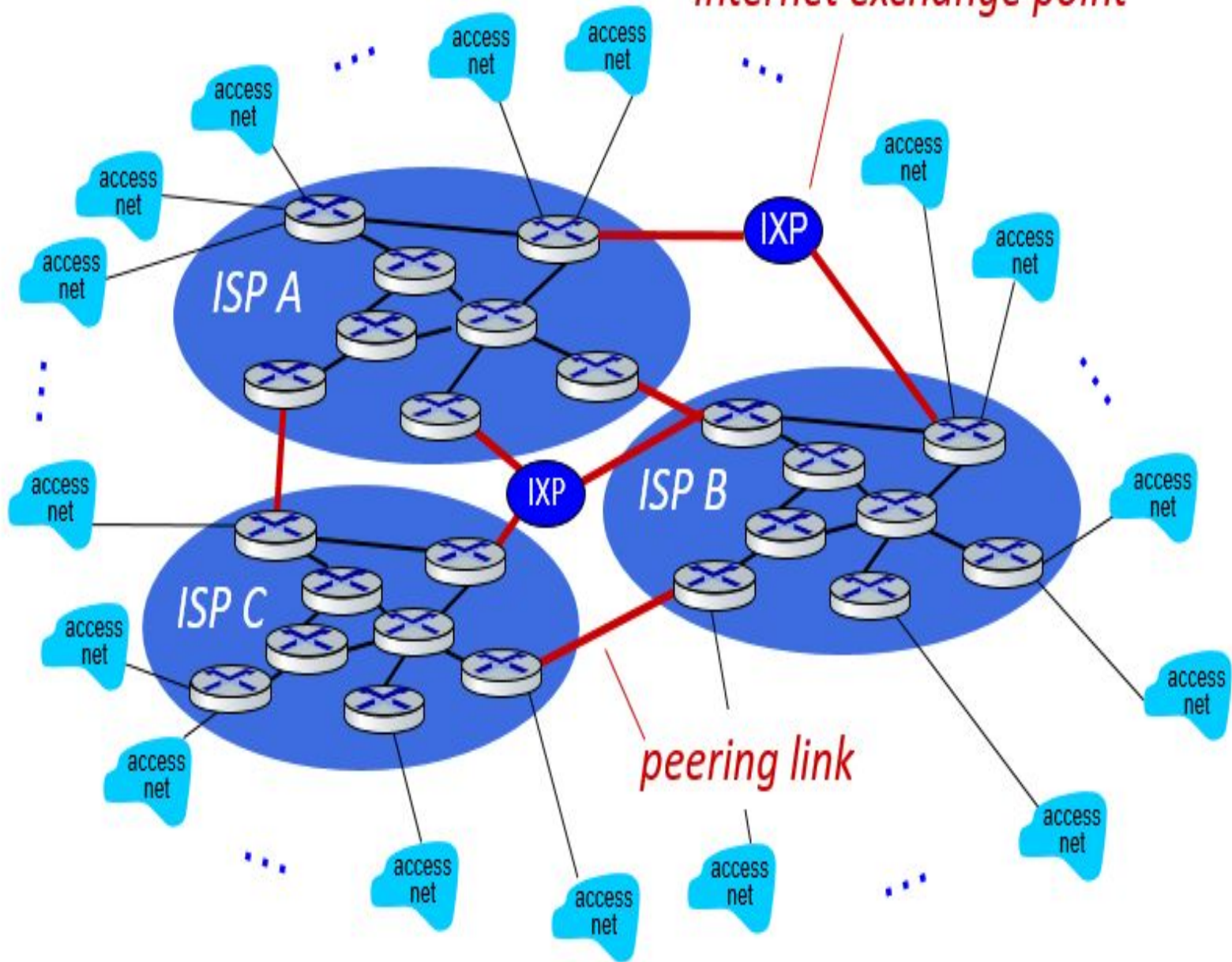
Any ISP (except for tier-1 ISPs) may choose to **multi-home**, that is, to connect to two or more provider ISPs.

So, for example, an access ISP may multi-home with two regional ISPs, or it may multi-home with two regional ISPs and also with a tier-1 ISP.

Similarly, a regional ISP may multi-home with multiple tier-1 ISPs.

When an ISP multi-homes, it can continue to send and receive packets into the Internet even if one of its providers has a failure.

Internet exchange point



As we just learned, customer ISPs pay their provider ISPs to obtain global Internet interconnectivity. The amount that a customer ISP pays a provider ISP reflects the amount of traffic it exchanges with the provider.

To reduce these costs, a pair of nearby ISPs at the same level of the hierarchy can **peer**, that is, they can directly connect their networks together so that all the traffic between them passes over the direct connection rather than through upstream intermediaries.

When two ISPs peer, it is typically settlement-free, that is, neither ISP pays the other.

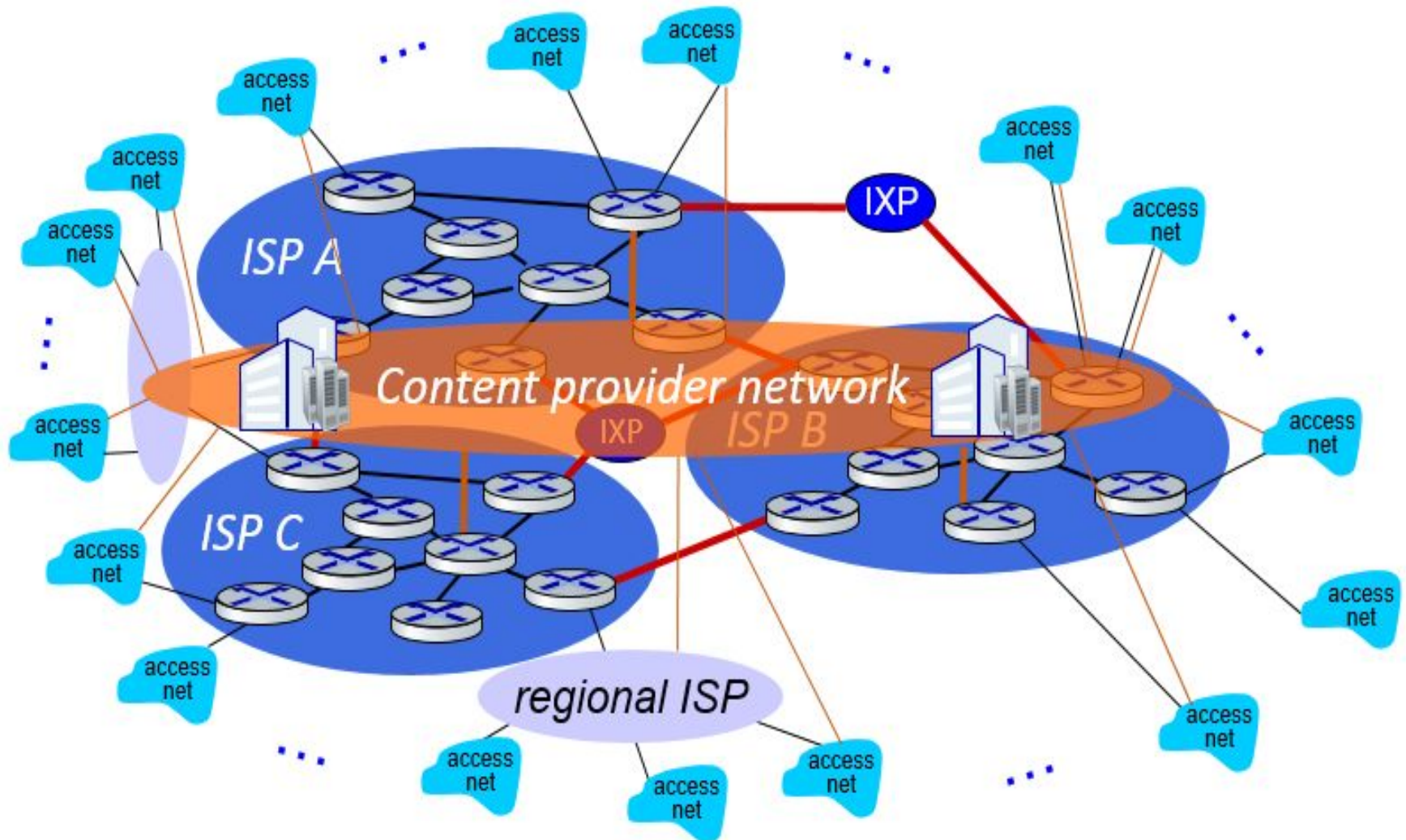
As noted earlier, tier-1 ISPs also peer with one another, settlement-free.

a third-party company can create an **Internet Exchange Point (IXP)**, which is a meeting point where multiple ISPs can peer together.

An IXP is typically in a stand-alone building with its own switches [Ager 2012].

We refer to this ecosystem—consisting of access ISPs, regional ISPs, tier-1 ISPs, PoPs, multi-homing, peering, and IXPs—as *Network Structure 4*.

We now finally arrive at *Network Structure 5*, which describes today's Internet. Network Structure 5, illustrated in Figure 1.15, builds on top of Network Structure 4 by adding **content-provider networks**.



Google is currently one of the leading examples of such a content-provider network.

As of this writing, it Google has 19 major data centers distributed across North America, Europe, Asia, South America, and Australia with each data center having tens or hundreds of thousands of servers.

The Google data centers are all interconnected via Google's private TCP/IP network, which spans the entire globe but is nevertheless separate from the public Internet.

Importantly, the Google private network only carries traffic to/from Google servers.

As shown in Figure 1.15, the Google private network attempts to “bypass” the upper tiers of the Internet by peering (settlement free) with lower-tier ISPs, either by directly connecting with them or by connecting with them at IXPs [Labovitz 2010].

However, because many access ISPs can still only be reached by transiting through tier-1 networks, the Google network also connects to tier-1 ISPs, and pays those ISPs for the traffic it exchanges with them.

By creating its own network, a content provider not only reduces its payments to upper-tier ISPs, but also has greater control of how its services are ultimately delivered to end users.

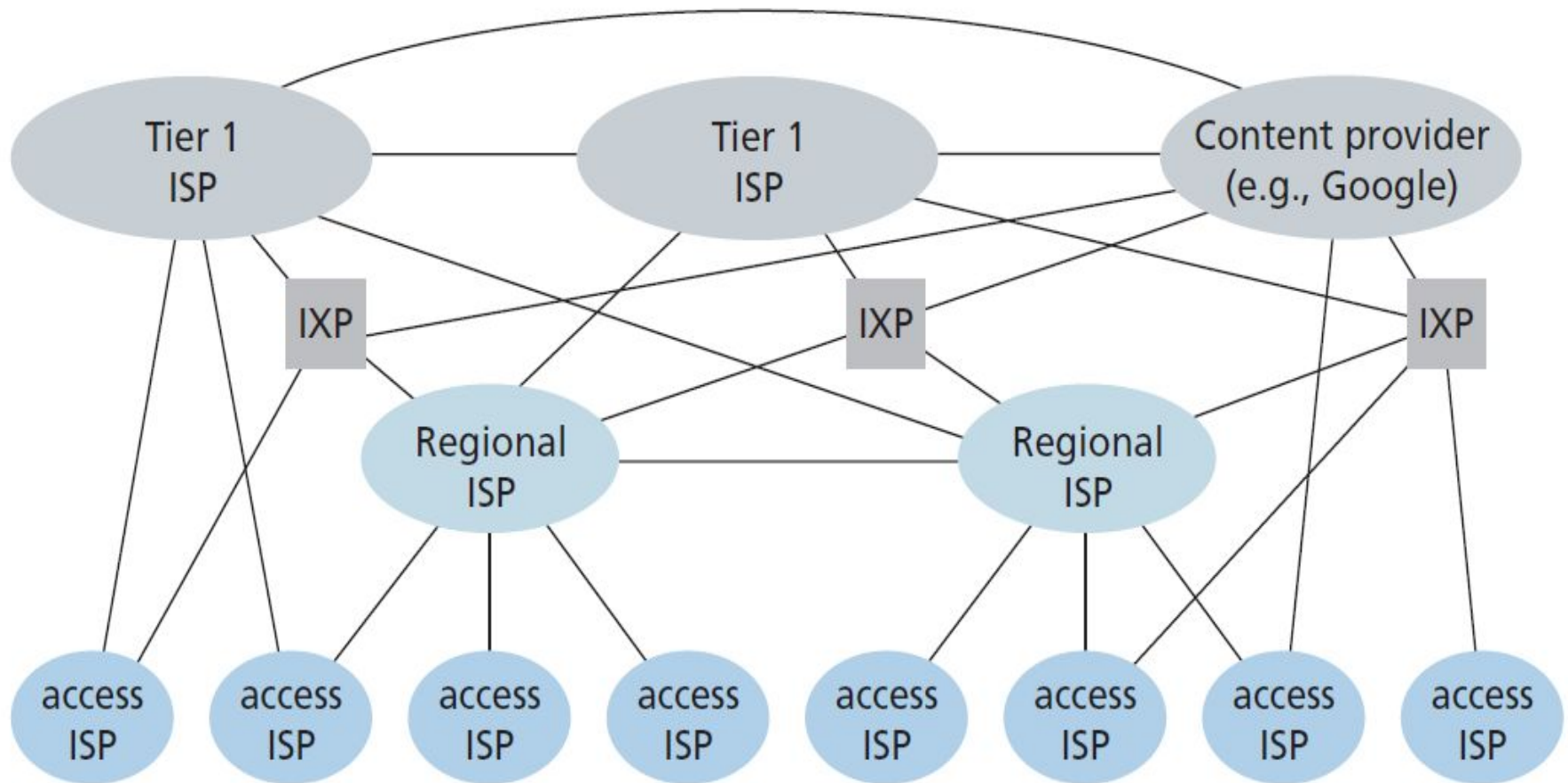


Figure 1.15 ♦ Interconnection of ISPs

1.4 Delay, Loss, and Throughput in Packet-Switched Networks

1.4.1 Overview of Delay in Packet-Switched Networks

Recall that a packet starts in a host (the source), passes through a series of routers, and ends its journey in another host (the destination).

As a packet travels from one node (host or router) to the subsequent node (host or router) along this path, the packet suffers from several types of delays at *each* node along the path. The most important of these delays are the **nodal processing delay**, **queuing delay**, **transmission delay**, and **propagation delay**; together, these delays accumulate to give a **total nodal delay**.

The performance of many Internet applications—such as search, Web browsing, e-mail, maps, instant messaging, and voice-over-IP—are greatly affected by network delays.

In order to acquire a deep understanding of packet switching and computer networks, we must understand the nature and importance of these delays.

Types of Delay

Let's explore these delays in the context of Figure 1.16.

As part of its end-to-end route between source and destination, a packet is sent from the upstream node through router A to router B.

Our goal is to characterize the nodal delay at router A.

Note that router A has an outbound link leading to router B.

This link is preceded by a queue (also known as a buffer).

When the packet arrives at router A from the upstream node, router A examines the packet's header to determine the appropriate outbound link for the packet and then directs the packet to this link.

In this example, the outbound link for the packet is the one that leads to router B.

A packet can be transmitted on a link only if there is no other packet currently being transmitted on the link and if there are no other packets preceding it in the queue; if the link is currently busy or if there are other packets already queued for the link, the newly arriving packet will then join the queue.

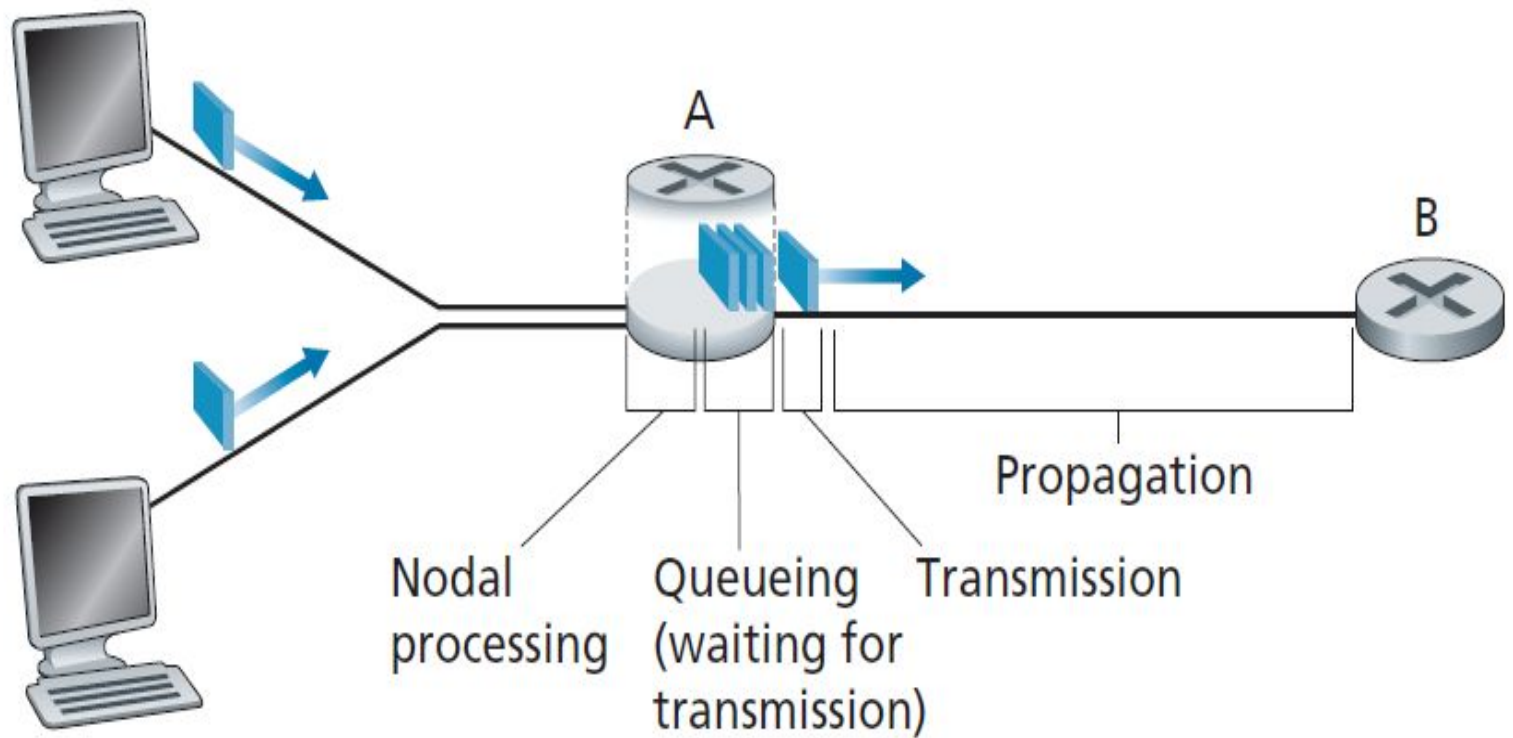


Figure 1.16 ♦ The nodal delay at router A

Processing Delay

The time required to examine the packet's header and determine where to direct the packet is part of the **processing delay**.

The processing delay can also include other factors, such as the time needed to check for bit-level errors in the packet that occurred in transmitting the packet's bits from the upstream node to router A.

Processing delays in high-speed routers are typically on the order of microseconds or less. After this nodal processing, the router directs the packet to the queue that precedes the link to router B.

Queuing Delay

At the queue, the packet experiences a **queuing delay** as it waits to be transmitted onto the link.

The length of the queuing delay of a specific packet will depend on the number of earlier-arriving packets that are queued and waiting for transmission onto the link.

If the queue is empty and no other packet is currently being transmitted, then our packet's queuing delay will be zero.

On the other hand, if the traffic is heavy and many other packets are also waiting to be transmitted, the queuing delay will be long.

Queuing delays can be on the order of microseconds to milliseconds in practice.

Transmission Delay

Assuming that packets are transmitted in a first-come-first-served manner, as is common in packet-switched networks, our packet can be transmitted only after all the packets that have arrived before it have been transmitted.

Denote the length of the packet by L bits, and denote the transmission rate of the link from router A to router B by R bits/sec.

For example, for a 10 Mbps Ethernet link, the rate is $R = 10$ Mbps; for a 100 Mbps Ethernet link, the rate is $R = 100$ Mbps.

The **transmission delay** is L/R .

This is the amount of time required to push (that is, transmit) all of the packet's bits into the link.

Transmission delays are typically on the order of microseconds to milliseconds in practice.

Propagation Delay

Once a bit is pushed into the link, it needs to propagate to router B.

The time required to propagate from the beginning of the link to router B is the **propagation delay**. The bit propagates at the propagation speed of the link.

The propagation speed depends on the physical medium of the link (that is, fiber optics, twisted-pair copper wire, and so on) and is in the range of

$$2 \cdot 10^8 \text{ meters/sec to } 3 \cdot 10^8 \text{ meters/sec}$$

which is equal to, or a little less than, the speed of light.

The propagation delay is the distance between two routers divided by the propagation speed.

That is, the propagation delay is d/s , where d is the distance between router A and router B and s is the propagation speed of the link.

Once the last bit of the packet propagates to node B, it and all the preceding bits of the packet are stored in router B.

The whole process then continues with router B now performing the forwarding.

In wide-area networks, propagation delays are on the order of milliseconds.

If we let d_{proc} , d_{queue} , d_{trans} , and d_{prop} denote the processing, queuing, transmission, and propagation delays, then the total nodal delay is given by

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

The contribution of these delay components can vary significantly.

For example, d_{prop} can be negligible (for example, a couple of microseconds) for a link connecting two routers on the same university campus; however, d_{prop} is hundreds of milliseconds for two routers interconnected by a geostationary satellite link, and can be the dominant term in d_{nodal} .

Similarly, d_{trans} can range from negligible to significant.

The processing delay, d_{proc} , is often negligible;

1.4.2 Queuing Delay and Packet Loss

The most complicated and interesting component of nodal delay is the queuing delay, d_{queue} .

Unlike the other three delays (namely, d_{proc} , d_{trans} , and d_{prop}), the queuing delay can vary from packet to packet.

For example, if 10 packets arrive at an empty queue at the same time, the first packet transmitted will suffer no queuing delay, while the last packet transmitted will suffer a relatively large queuing delay (while it waits for the other nine packets to be transmitted).

When is the queuing delay large and when is it insignificant? The answer to this question depends on the rate at which traffic arrives at the queue, the transmission rate of the link, and the nature of the arriving traffic, that is, whether the traffic arrives periodically or arrives in bursts.

To gain some insight here, let a denote the average rate at which packets arrive at the queue (a is in units of packets/sec).

Recall that R is the transmission rate; that is, it is the rate (in bits/sec) at which bits are pushed out of the queue.

The ratio La/R , called the **traffic intensity**, often plays an important role in estimating the extent of the queuing delay. estimating the extent of the queuing delay.

If $La/R > 1$, then the average rate at which bits arrive at the queue exceeds the rate at which the bits can be transmitted from the queue.

In this unfortunate situation, the queue will tend to increase without bound and the queuing delay will approach infinity! Therefore, one of the golden rules in traffic engineering is:

Design your system so that the traffic intensity is no greater than 1.

Now consider the case

$$La/R \leq 1$$

For example, if packets arrive periodically—that is, one packet arrives every L/R seconds—then every packet will arrive at an empty queue and there will be no queuing delay. On the other hand, if packets arrive in bursts but periodically, there can be a significant average queuing delay.

For example, suppose N packets arrive simultaneously every $(L/R)N$ seconds.

Then the first packet transmitted has no queuing delay; the second packet transmitted has a queuing delay of L/R seconds; and more generally, the n th packet transmitted has a queuing delay of $(n - 1)L/R$ seconds.

In particular, if the traffic intensity is close to zero, then packet arrivals are few and far between and it is unlikely that an arriving packet will find another packet in the queue. Hence, the average queuing delay will be close to zero.

One important aspect of Figure 1.18 is the fact that as the traffic intensity approaches 1, the average queuing delay increases rapidly.

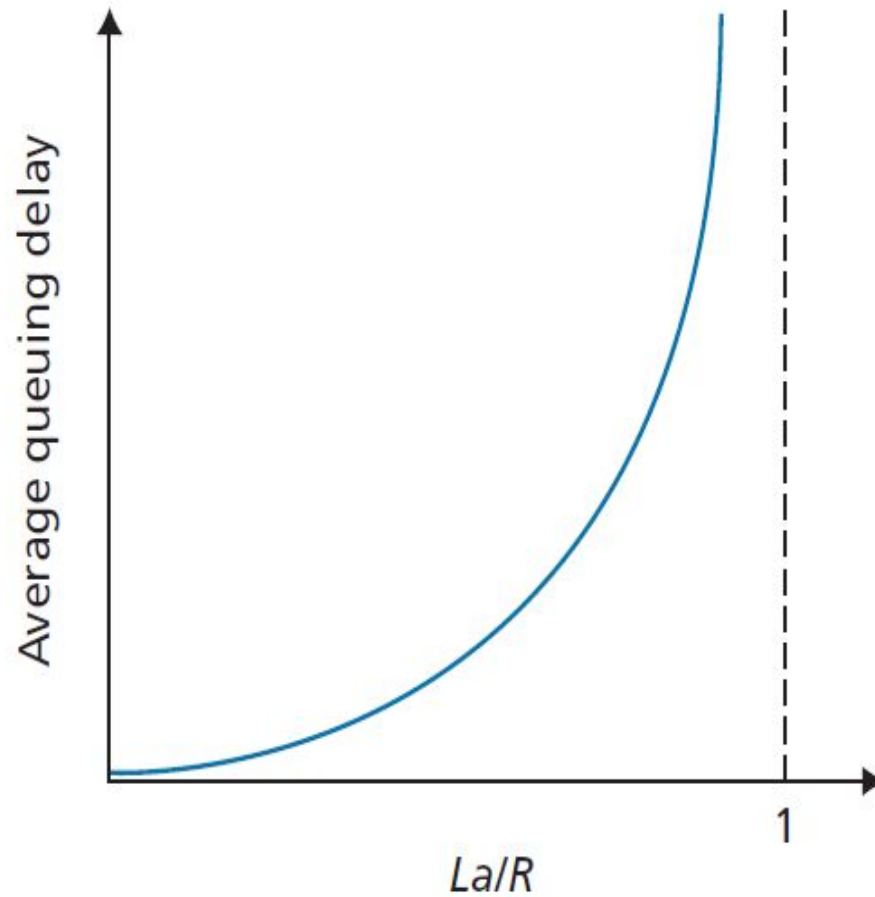


Figure 1.18 ♦ Dependence of average queuing delay on traffic intensity

Packet Loss

In our discussions above, we have assumed that the queue is capable of holding an infinite number of packets.

In reality a queue preceding a link has finite capacity, although the queuing capacity greatly depends on the router design and cost.

Instead, a packet can arrive to find a full queue.

With no place to store such a packet, a router will **drop** that packet; that is, the packet will be **lost**.

The fraction of lost packets increases as the traffic intensity increases.

Therefore, performance at a node is often measured not only in terms of delay, but also in terms of the probability of packet loss.

As we'll discuss in the subsequent chapters, a lost packet may be retransmitted on an end-to-end basis in order to ensure that all data are eventually transferred from source to destination.

Our discussion up to this point has focused on the nodal delay, that is, the delay at a single router.

Let's now consider the total delay from source to destination.

To get a handle on this concept, suppose there are $N - 1$ routers between the source host and the destination host.

Let's also suppose for the moment that the network is uncongested (so that queuing delays are negligible), the processing delay at each router and at the source host is d_{proc} , the transmission rate out of each router and out of the source host is R bits/sec, and the propagation on each link is d_{prop} .

The nodal delays accumulate and give an end-to-end delay,

$$d_{\text{end-end}} = N (d_{\text{proc}} + d_{\text{trans}} + d_{\text{prop}}) \quad (1.2)$$

where, once again, $d_{\text{trans}} = L/R$, where L is the packet size.

Note that Equation 1.2 is a generalization of Equation 1.1, which did not take into account processing and propagation delays.

Traceroute

To get a hands-on feel for end-to-end delay in a computer network, we can make use of the Traceroute program.

Traceroute is a simple program that can run in any Internet host.

When the user specifies a destination hostname, the program in the source host sends multiple, special packets toward that destination.

End System, Application, and Other Delays

In addition to processing, transmission, and propagation delays, there can be additional significant delays in the end systems.

1.4.4 Throughput in Computer Networks

In addition to delay and packet loss, another critical performance measure in computer networks is end-to-end throughput.

To gain further insight into the important concept of throughput, let's consider a few examples.

Figure 1.19(a) shows two end systems, a server and a client, connected by two communication links and a router.

Consider the throughput for a file transfer from the server to the client.

Let R_s denote the rate of the link between the server and the router; and R_c denote the rate of the link between the router and the client.

Suppose that the only bits being sent in the entire network are those from the server to the client.

We now ask, in this ideal scenario, what is the server-to-client throughput? To answer this question,

Clearly, the server cannot pump bits through its link at a rate faster than R_s bps; and the router cannot forward bits at a rate faster than R_c bps.

If $R_s < R_c$, then the bits pumped by the server will “flow” right through the router and arrive at the client at a rate of R_s bps, giving a throughput of R_s bps.

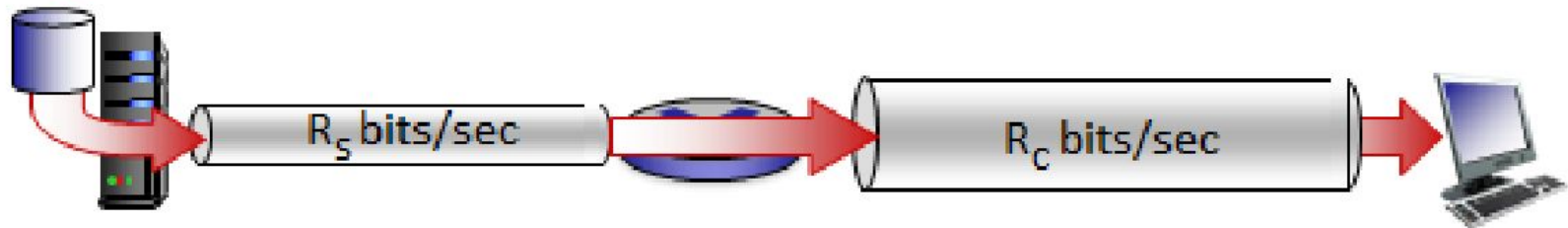
If, on the other hand, $R_c < R_s$, then the router will not be able to forward bits as quickly as it receives them.

In this case, bits will only leave the router at rate R_c , giving an end-to-end throughput of R_c . (Note also that if bits continue to arrive at the router at rate R_s , and continue to leave the router at R_c , the backlog of bits at the router waiting for transmission to the client will grow and grow—a most undesirable situation!)

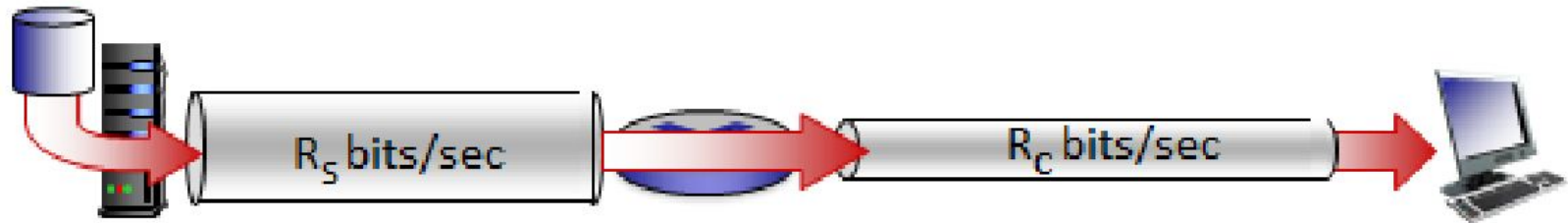
Thus, for this simple two-link network, the throughput is $\min\{R_c, R_s\}$, that is, it is the transmission rate of the **bottleneck link**.

Having determined the throughput, we can now approximate the time it takes to transfer a large file of F bits from server to client as $F/\min\{R_s, R_c\}$.

$R_s < R_c$ What is average end-end throughput?



$R_s > R_c$ What is average end-end throughput?



bottleneck link

link on end-end path that constrains end-end throughput

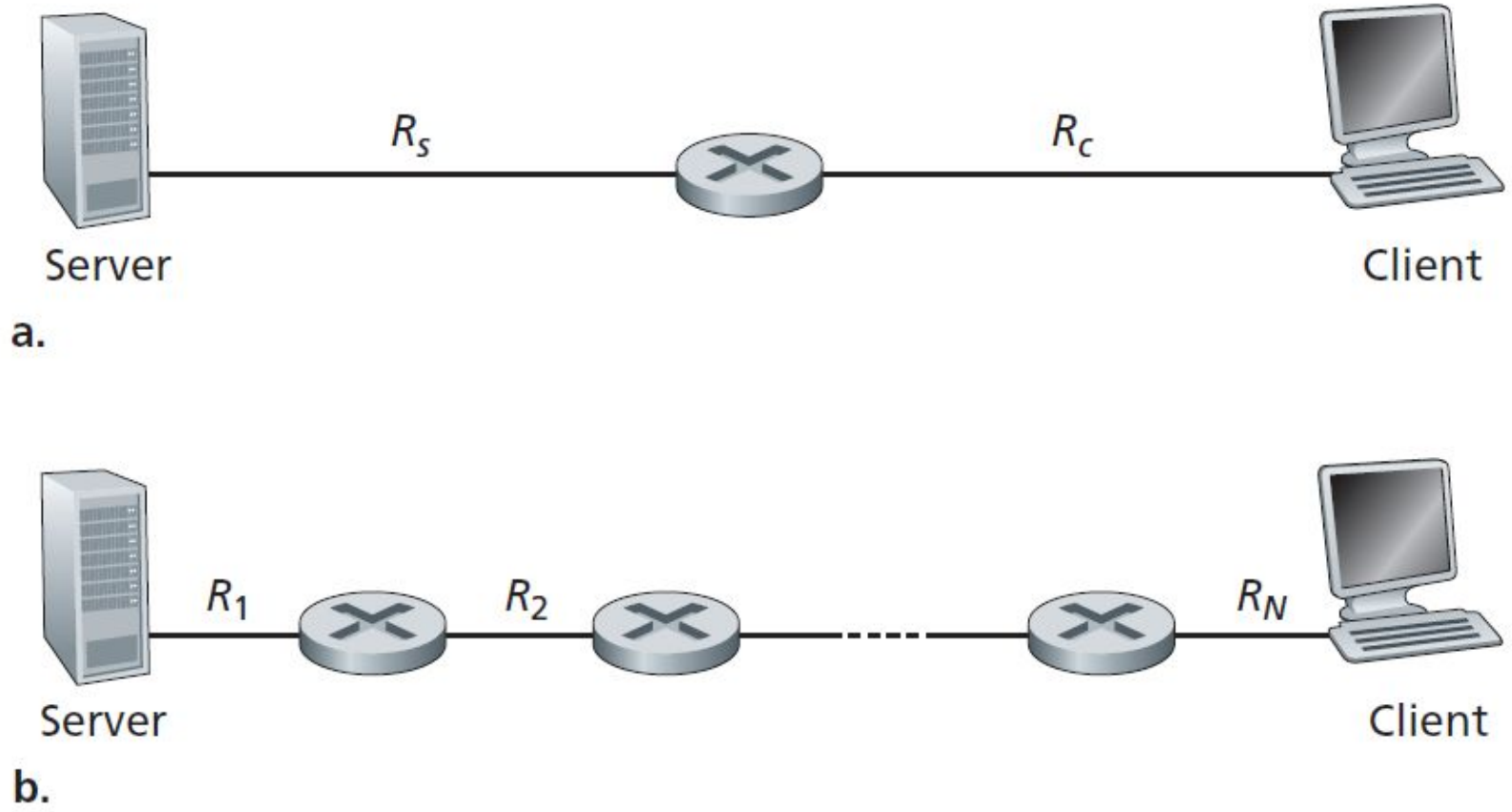


Figure 1.19 ♦ Throughput for a file transfer from server to client

Figure 1.19(b) now shows a network with N links between the server and the client, with the transmission rates of the N links being $R_1, R_2, \dots, R_N$.

Applying the same analysis as for the two-link network, we find that the throughput for a file transfer from server to client is $\min\{R_1, R_2, \dots, R_N\}$, which is once again the transmission rate of the bottleneck link along the path between server and client.

Now consider another example motivated by today's Internet.

Figure 1.20(a) shows two end systems, a server and a client, connected to a computer network.

Consider the throughput for a file transfer from the server to the client.

The server is connected to the network with an access link of rate R_s and the client is connected to the network with an access link of rate R_c .

Now suppose that all the links in the core of the communication network have very high transmission rates, much higher than R_s and R_c .

Indeed, today, the core of the Internet is over-provisioned with high speed links that experience little congestion.

Also suppose that the only bits being sent in the entire network are those from the server to the client. Because the core of the computer network is like a wide pipe in this example, the rate at which bits can flow from source to destination is again the minimum of R_s and R_c , that is, throughput = $\min\{R_s, R_c\}$.

Therefore, the constraining factor for throughput in today's Internet is typically the access network.

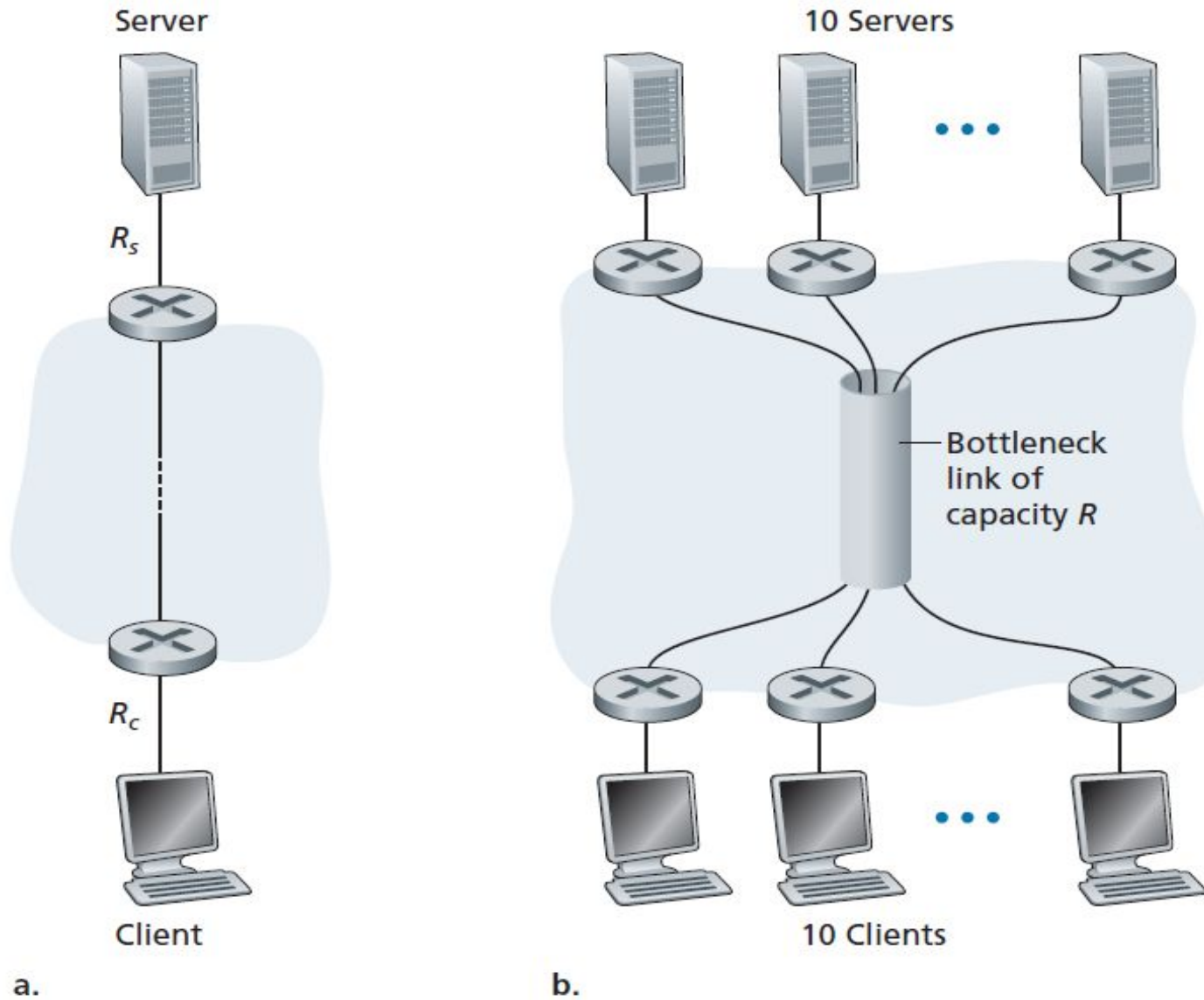


Figure 1.20 ♦ End-to-end throughput: (a) Client downloads a file from server; (b) 10 clients downloading with 10 servers

For a final example, consider Figure 1.20(b) in which there are 10 servers and 10 clients connected to the core of the computer network.

In this example, there are 10 simultaneous downloads taking place, involving 10 client-server pairs.

Suppose that these 10 downloads are the only traffic in the network at the current time.

As shown in the figure, there is a link in the core that is traversed by all 10 downloads.

Denote R for the transmission rate of this link R .

Let's suppose that all server access links have the same rate R_s , all client access links have the same rate R_c , and the transmission rates of all the links in the core—except the one common link of rate R —are much larger than R_s , R_c , and R .

Now we ask, what are the throughputs of the downloads? Clearly, if the rate of the common link, R , is large—say a hundred times larger than both R_s and R_c —then the throughput for each download will once again be $\min\{R_s, R_c\}$.

But what if the rate of the common link is of the same order as R_s and R_c ? What will the throughput be in this case? Let's take a look at a specific example.

Suppose $R_s = 2$ Mbps, $R_c = 1$ Mbps, $R = 5$ Mbps, and the common link divides its transmission rate equally among the 10 downloads.

Then the bottleneck for each download is no longer in the access network, but is now instead the shared link in the core, which only provides each download with 500 kbps of throughput.

Thus, the end-to-end throughput for each download is now reduced to 500 kbps.

The examples in Figure 1.19 and Figure 1.20(a) show that throughput depends on the transmission rates of the links over which the data flows.

We saw that when there is no other intervening traffic, the throughput can simply be approximated as the minimum transmission rate along the path between source and destination.

The example in Figure 1.20(b) shows that more generally the throughput depends not only on the transmission rates of the links along the path, but also on the intervening traffic.

In particular, a link with a high transmission rate may nonetheless be the bottleneck link for a file transfer if many other data flows are also passing through that link.

1.5 Protocol Layers and Their Service Models

1.5.1 Layered Architecture

Protocol Layering

Let's now turn our attention to network protocols.

To provide structure to the design of network protocols, network designers organize protocols—and the network hardware and software that implement the protocols—in **layers**.

Each protocol belongs to one of the layers,

We are again interested in the **services** that a layer offers to the layer above—the so-called **service model** of a layer.

each layer provides its service by

- (1) performing certain actions within that layer and by
- (2) using the services of the layer directly below it.

A protocol layer can be implemented in software, in hardware, or in a combination of the two.

Application-layer protocols—such as HTTP and SMTP—are almost always implemented in software in the end systems; so are transport-layer protocols.

Because the physical layer and data link layers are typically implemented in a network interface card (for example, Ethernet or WiFi interface cards) associated with a given link.

The network layer is often a mixed implementation of hardware and software.

a layer n protocol *distributed* among the end systems, packet switches, and other components that make up the network.

We mention, however, that some researchers and networking engineers are vehemently opposed to layering [Wakeman 1992].

One potential drawback of layering is that one layer may duplicate lower-layer functionality.

A second potential drawback is that functionality at one layer may need information that is present only in another layer; this violates the goal of separation of layers.

When taken together, the protocols of the various layers are called the **protocol stack**.

The Internet protocol stack consists of five layers: the physical, link, network, transport, and application layers, as shown in Figure 1.23.

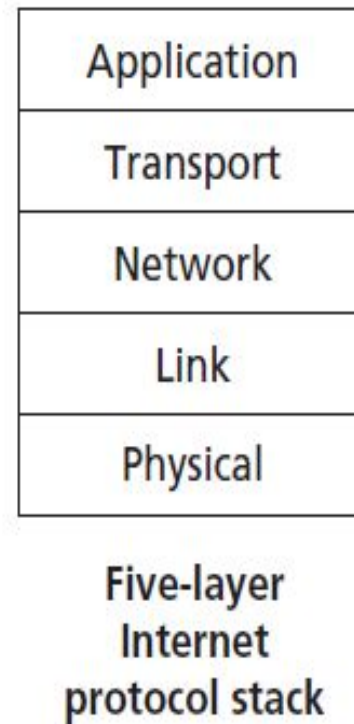


Figure 1.23 ♦ The Internet protocol stack

Application Layer

The application layer is where network applications and their application-layer protocols reside.

The Internet's application layer includes many protocols, such as the HTTP protocol (which provides for Web document request and transfer), SMTP (which provides for the transfer of e-mail messages), and FTP (which provides for the transfer of files between two end systems).

We'll see that certain network functions, such as the translation of human-friendly names for Internet end systems like `www.ietf.org` to a 32-bit network address, are also done with the help of a specific application-layer protocol, namely, the domain name system (DNS).

An application-layer protocol is distributed over multiple end systems, with the application in one end system using the protocol to exchange packets of information with the application in another end system.

We'll refer to this packet of information at the application layer as a **message**.

Transport Layer

The Internet's transport layer transports application-layer messages between application endpoints.

In the Internet, there are two transport protocols, TCP and UDP, either of which can transport application-layer messages.

TCP provides a connection-oriented service to its applications.

This service includes guaranteed delivery of application-layer messages to the destination and flow control (that is, sender/receiver speed matching).

TCP also breaks long messages into shorter segments and provides a congestion-control mechanism, so that a source throttles its transmission rate when the network is congested.

The UDP protocol provides a connectionless service to its applications.

This is a no-frills service that provides no reliability, no flow control, and no congestion control.

In this book, we'll refer to a transport-layer packet as a **segment**.

Network Layer

The Internet's network layer is responsible for moving network-layer packets known as **datagrams** from one host to another.

The Internet transport-layer protocol (TCP or UDP) in a source host passes a transport-layer segment and a destination address to the network layer

The network layer then provides the service of delivering the segment to the transport layer in the destination host.

The Internet's network layer includes the celebrated IP protocol, which defines the fields in the datagram as well as how the end systems and routers act on these fields.

There is only one IP protocol, and all Internet components that have a network layer must run the IP protocol.

The Internet's network layer also contains routing protocols that determine the routes that datagrams take between sources and destinations.

The Internet has many routing protocols.

Although the network layer contains both the IP protocol and numerous routing protocols, it is often simply referred to as the IP layer, reflecting the fact that IP is the glue that binds the Internet together.

Link Layer

To move a packet from one node (host or router) to the next node in the route, the network layer relies on the services of the link layer.

In particular, at each node, the network layer passes the datagram down to the link layer, which delivers the datagram to the next node along the route.

At this next node, the link layer passes the datagram up to the network layer.

The services provided by the link layer depend on the specific link-layer protocol that is employed over the link.

For example, some link-layer protocols provide reliable delivery, from transmitting node, over one link, to receiving node.

Note that this reliable delivery service is different from the reliable delivery service of TCP, which provides reliable delivery from one end system to another.

Examples of link-layer protocols include Ethernet, WiFi, and the cable access network's DOCSIS protocol.

As datagrams typically need to traverse several links to travel from source to destination, a datagram may be handled by different link-layer protocols at different links along its route. For example, a datagram may be handled by Ethernet on one link and by PPP on the next link. The network layer will receive a different service from each of the different link-layer protocols.

In this book, we'll refer to the link-layer packets as **frames**.

Physical Layer

While the job of the link layer is to move entire frames from one network element to an adjacent network element, the job of the physical layer is to move the *individual bits* within the frame from one node to the next.

The protocols in this layer are again link dependent and further depend on the actual transmission medium of the link (for example, twisted-pair copper wire, single-mode fiber optics).

For example, Ethernet has many physical-layer protocols: one for twisted-pair copper wire, another for coaxial cable, another for fiber, and so on.

In each case, a bit is moved across the link in a different way.

1.5.2 Encapsulation

Figure 1.24 shows the physical path that data takes down a sending end system's protocol stack, up and down the protocol stacks of an intervening link-layer switch and router, and then up the protocol stack at the receiving end system.

As we discuss later in this book, routers and link-layer switches are both packet switches. Similar to end systems, routers and link-layer switches organize their networking hardware and software into layers.

But routers and link-layer switches do not implement *all* of the layers in the protocol stack; they typically implement only the bottom layers.

As shown in Figure 1.24, link-layer switches implement layers 1 and 2; routers implement layers 1 through 3.

This means, for example, that Internet routers are capable of implementing the IP protocol (a layer 3 protocol), while link-layer switches are not.

We'll see later that while link-layer switches do not recognize IP addresses, they are capable of recognizing layer 2 addresses, such as Ethernet addresses.

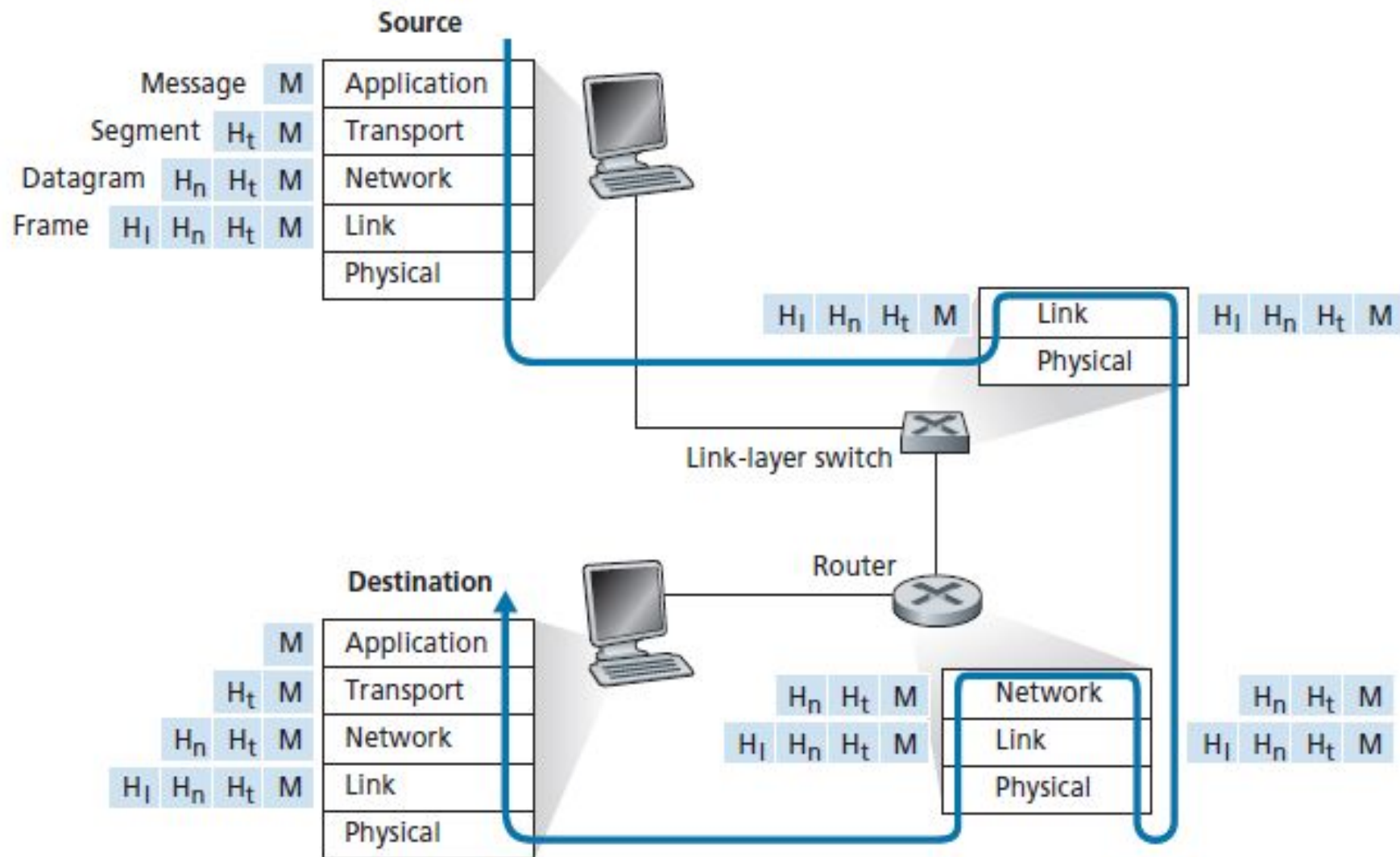


Figure 1.24 ♦ Hosts, routers, and link-layer switches; each contains a different set of layers, reflecting their differences in functionality

Note that hosts implement all five layers; this is consistent with the view that the Internet architecture puts much of its complexity at the edges of the network.

Figure 1.24 also illustrates the important concept of **encapsulation**.

At the sending host, an **application-layer message** (M in Figure 1.24) is passed to the transport layer.

In the simplest case, the transport layer takes the message and appends additional information (so-called transport-layer header information, H, in Figure 1.24) that will be used by the receiver-side transport layer.

The application-layer message and the transport-layer header information together constitute the **transport-layer segment**.

The transport-layer segment thus encapsulates the application-layer message.

The added information might include information allowing the receiver-side transport layer to deliver the message up to the appropriate application, and error-detection bits that allow the receiver to determine whether bits in the message have been changed in route.

The transport layer then passes the segment to the network layer, which adds network-layer header information (H_n in Figure 1.24) such as source and destination end system addresses, creating a **network-layer datagram**.

The datagram is then passed to the link layer, which (of course!) will add its own link-layer header information and create a **link-layer frame**.

Thus, we see that at each layer, a packet has two types of fields: header fields and a **payload field**. The payload is typically a packet from the layer above.

The process of encapsulation can be more complex than that described above.

1.6 Networks Under Attack

The field of network security is about how the bad guys can attack computer networks and about how we can defend networks against those attacks, or better yet, design new architectures that are immune to such attacks in the first place.

Given the frequency and variety of existing attacks as well as the threat of new and more destructive future attacks, network security has become a central topic in the field of computer networking.

The Bad Guys Can Put Malware into Your Host Via the Internet

We attach devices to the Internet because we want to receive/send data from/to the Internet. This includes all kinds of good stuff, including Instagram posts, Internet search results, streaming music, video conference calls, streaming movies, and so on.

But, unfortunately, along with all that good stuff comes malicious stuff—collectively known as **malware**—that can also enter and infect our devices.

Once malware infects our device it can do all kinds of devious things, including deleting our files and installing spyware that collects our private information, such as social security numbers, passwords, and keystrokes, and then sends this (over the Internet, of course!) back to the bad guys.

Our compromised host may also be enrolled in a network of thousands of similarly compromised devices, collectively known as a **botnet**, which the bad guys control and leverage for spam e-mail distribution or distributed denial-of-service attacks (soon to be discussed) against targeted hosts.

Much of the malware out there today is **self-replicating**: once it infects one host, from that host it seeks entry into other hosts over the Internet, and from the newly infected hosts, it seeks entry into yet more hosts.

In this manner, self- replicating malware can spread exponentially fast.

The Bad Guys Can Attack Servers and Network Infrastructure

Another broad class of security threats are known as **denial-of-service (DoS) attacks**.

As the name suggests, a DoS attack renders a network, host, or other piece of infrastructure unusable by legitimate users.

Web servers, e-mail servers, DNS servers (discussed in Chapter 2), and institutional networks can all be subject to DoS attacks.

Most Internet DoS attacks fall into one of three categories:

- *Vulnerability attack.*
- *Bandwidth flooding.*
- *Connection flooding.*

Let's now explore the bandwidth-flooding attack in more detail.

it's evident that if the server has an access rate of R bps, then the attacker will need to send traffic at a rate of approximately R bps to cause damage.

If R is very large, a single attack source may not be able to generate enough traffic to harm the server.

In a **distributed DoS (DDoS)** attack, illustrated in Figure 1.25, the attacker controls multiple sources and has each source blast traffic at the target.

With this approach, the aggregate traffic rate across all the controlled sources needs to be approximately R to cripple the service.

DDoS attacks leveraging botnets with thousands of comprised hosts are a common occurrence today [DAM 2020].

DDoS attacks are much harder to detect and defend against than a DoS attack from a single host.

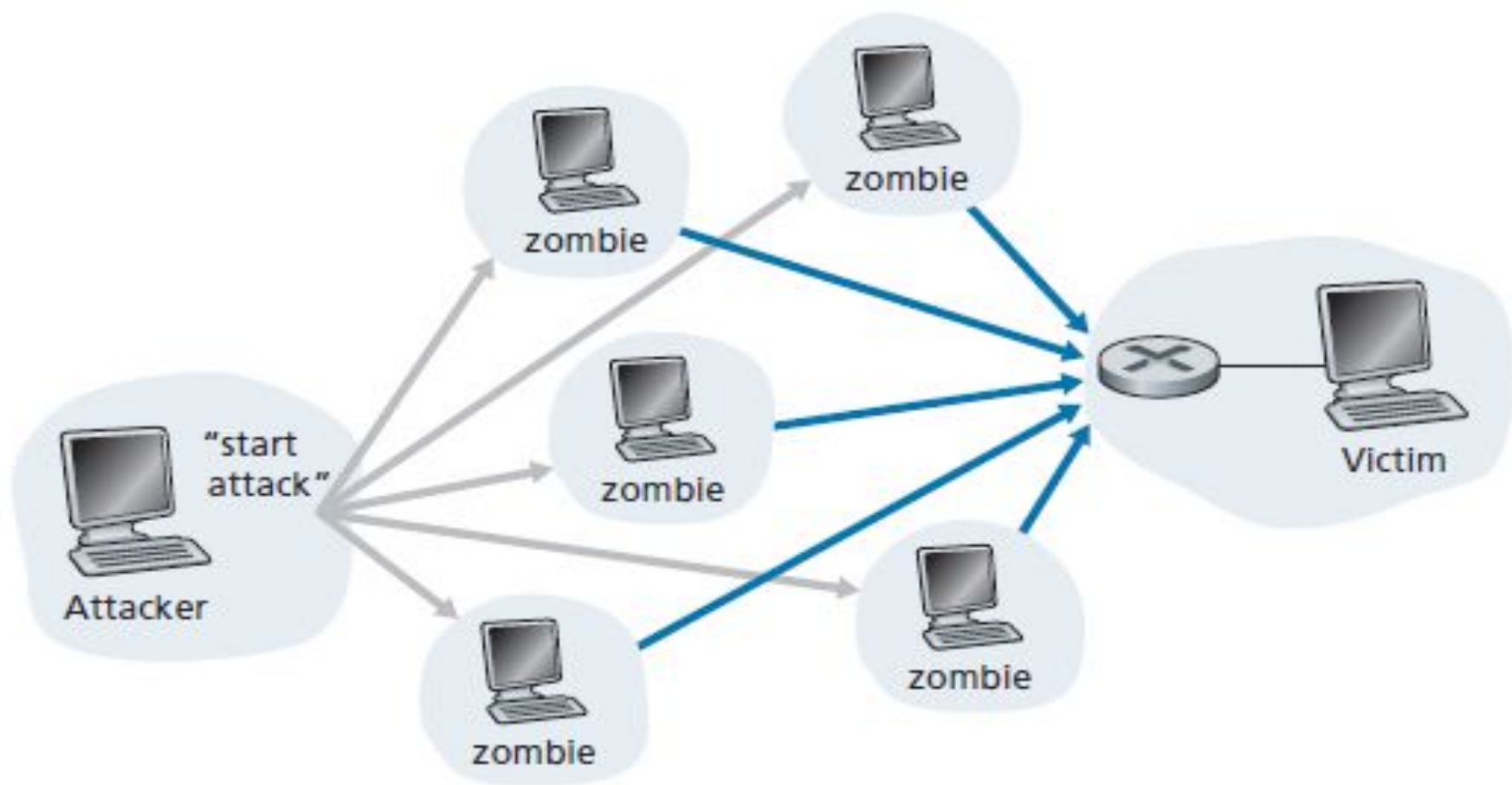


Figure 1.25 ♦ A distributed denial-of-service attack

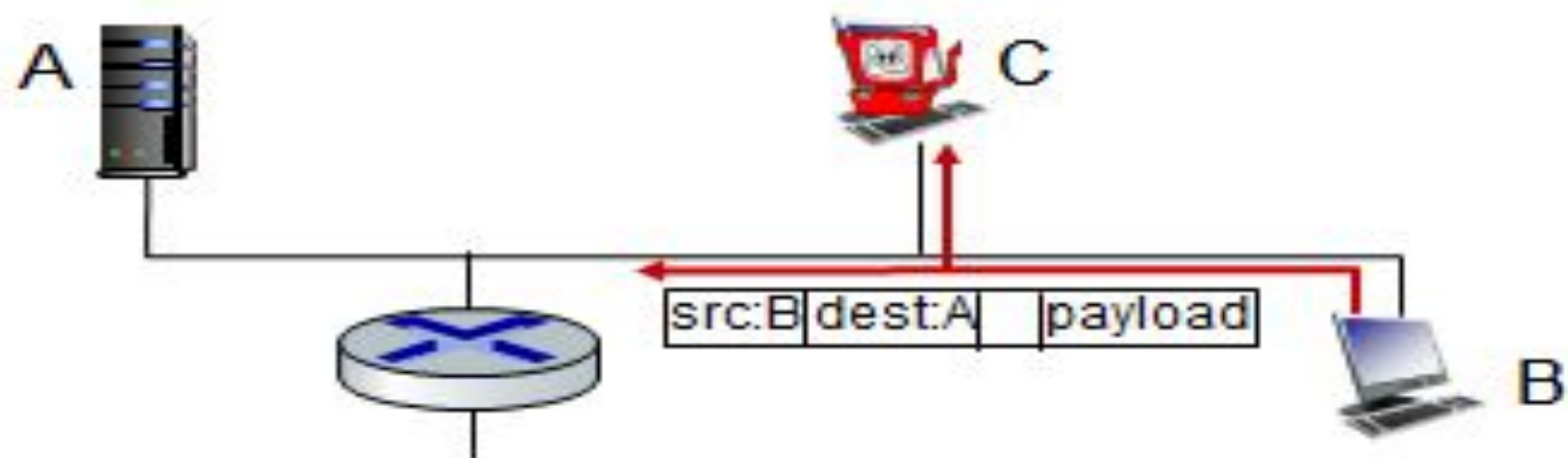
The Bad Guys Can Sniff Packets

Many users today access the Internet via wireless devices, such as WiFi-connected laptops or handheld devices with cellular Internet connections

placing a passive receiver in the vicinity of the wireless transmitter, that receiver can obtain a copy of every packet that is transmitted! These packets can contain all kinds of sensitive information, including passwords, social security numbers, trade secrets, and private personal messages.

A passive receiver that records a copy of every packet that flies by is called a **packet sniffer**.

Sniffers can be deployed in wired environments as well



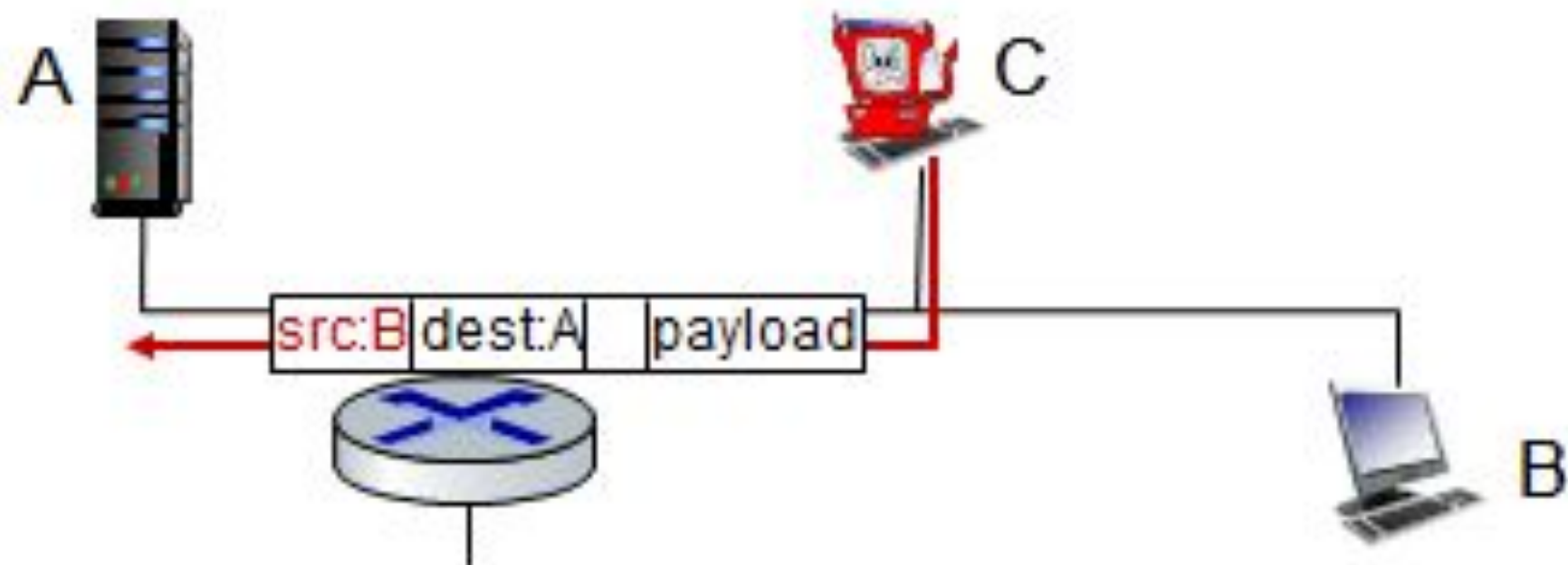
Sniffed packets can then be analyzed offline for sensitive information.

Packet-sniffing software is freely available at various Web sites and as commercial products.

Because packet sniffers are passive—that is, they do not inject packets into the channel—they are difficult to detect.

So, when we send packets into a wireless channel, we must accept the possibility that some bad guy may be recording copies of our packets.

As you may have guessed, some of the best defenses against packet sniffing involve cryptography.



The Bad Guys Can Masquerade as Someone You Trust

It is surprisingly easy to create a packet with an arbitrary source address, packet content, and destination address and then transmit this hand-crafted packet into the Internet, which will dutifully forward the packet to its destination.

Imagine the unsuspecting receiver (say an Internet router) who receives such a packet, takes the (false) source address as being truthful, and then performs some command embedded in the packet's contents (say modifies its forwarding table).

The ability to inject packets into the Internet with a false source address is known as **IP spoofing**, and is but one of many ways in which one user can masquerade as another user.

To solve this problem, we will need *end-point authentication*, that is, a mechanism that will allow us to determine with certainty if a message originates from where we think it does.

In closing this section, it's worth considering how the Internet got to be such an insecure place in the first place.

The answer, in essence, is that the Internet was originally designed to be that way, based on the model of “a group of mutually trusting users attached to a transparent network” [Blumenthal 2001]—a model in which (by definition) there is no need for security.

Many aspects of the original Internet architecture deeply reflect this notion of mutual trust.

But today's Internet certainly does not involve “mutually trusting users.”

Nonetheless, today's users still need to communicate when they don't necessarily trust each other, may wish to communicate anonymously, may communicate indirectly through third parties (e.g., Web caches, which we'll study in Chapter 2, or mobility-assisting agents, which we'll study in Chapter 7), and may distrust the hardware, software, and even the air through which they communicate.

We should keep in mind that communication among mutually trusted users is the exception rather than the rule.

1.7 History of Computer Networking and the Internet

1.7.1 The Development of Packet Switching: 1961–1972

The field of computer networking and today's Internet trace their beginnings back to the early 1960s, when the telephone network was the world's dominant communication network.

Given the increasing importance of computers in the early 1960s and the advent of timeshared computers, it was perhaps natural to consider how to hook computers together so that they could be shared among geographically distributed users.

Three research groups around the world, each unaware of the others' work [Leiner 1998], began inventing packet switching as an efficient and robust alternative to circuit switching.

The work at MIT, Rand, and the NPL laid the foundations for today's Internet.

an overall plan for the ARPAnet [Roberts 1967], the first packet-switched computer network and a direct ancestor of today's public Internet.

By 1972, ARPAnet had grown to approximately 15 nodes

The first host-to-host protocol between ARPAnet end systems, known as the network-control protocol (NCP),

the first e-mail program in 1972.

1.7.2 Proprietary Networks and Internetworking:1972–1980

The number of networks was growing.

The three key Internet protocols that we see today—TCP, UDP, and IP—were conceptually in place by the end of the 1970s.

1.7.3 A Proliferation of Networks: 1980–1990

By the end of the 1970s, approximately two hundred hosts were connected to the ARPAnet. By the end of the 1980s the number of hosts connected to the public Internet, a confederation of networks looking much like today's Internet, would reach a hundred thousand.

January 1, 1983 saw the official deployment of TCP/IP as the new standard host protocol for ARPAnet (replacing the NCP protocol).

The transition [RFC 801] from NCP to TCP/IP was a flag day event—all hosts were required to transfer over to TCP/IP as of that day.

The DNS, used to map between a human-readable Internet name (for example, `gaia.cs.umass.edu`) and its 32-bit IP address, was also developed [RFC 1034].

Paralleling this development of the ARPAnet (which was for the most part a US effort), in the early 1980s the French launched the Minitel project, an ambitious plan to bring data networking into everyone's home.

The Minitel was in a large proportion of French homes 10 years before most Americans had ever heard of the Internet.

1.7.4 The Internet Explosion: The 1990s

The main event of the 1990s was to be the emergence of the World Wide Web application, which brought the Internet into the homes and businesses of millions of people worldwide.

Berners-Lee and his associates developed initial versions of HTML, HTTP, a Web server, and a browser—the four key components of the Web.

Around the end of 1993 there were about two hundred Web servers in operation

At about this time several researchers were developing Web browsers with GUI interfaces

By 1995, university students were using Netscape browsers to surf the Web on a daily basis. At about this time companies—big and small—began to operate Web servers and transact commerce over the Web.

In 1996, Microsoft started to make browsers, which started the browser war between Netscape and Microsoft, which Microsoft won a few years later [Cusumano 1998].

The second half of the 1990s was a period of tremendous growth and innovation for the Internet, with major corporations and thousands of startups creating Internet products and services.

By the end of the millennium the Internet was supporting hundreds of popular applications, including four killer applications:

- E-mail, including attachments and Web-accessible e-mail
- The Web, including Web browsing and Internet commerce
- Instant messaging, with contact lists
- Peer-to-peer file sharing of MP3s, pioneered by Napster

a number of companies emerged as big winners in the Internet space, including Microsoft, Cisco, Yahoo, eBay, Google, and Amazon.

1.7.5 The New Millennium

- Since the beginning of the millennium, we have been seeing aggressive deployment of broadband Internet access to homes—not only cable modems and DSL but also fiber to the home, and now 5G fixed wireless

This high-speed Internet access has set the stage for a wealth of video applications, including the distribution of user-generated video (for example, YouTube), on-demand streaming of movies and television shows (e.g., Netflix), and multi-person video conference (e.g., Skype, Facetime, and Google Hangouts).

- The increasing ubiquity of high-speed wireless Internet access is not only making it possible to remain constantly connected while on the move,

The number of wireless devices connecting to the Internet surpassed the number of wired devices in 2011.

This high-speed wireless access has set the stage for the rapid emergence of hand-held computers (iPhones, Androids, iPads, and so on), which enjoy constant and untethered access to the Internet.

- Online social networks—such as Facebook, Instagram, Twitter, and WeChat (hugely popular in China)—have created massive people networks on top of the Internet. Many of these social networks are extensively used for messaging as well as photo sharing. Many Internet users today “live” primarily within one or more social networks.

online service providers, such as Google and Microsoft, have deployed their own extensive private networks, which not only connect together their globally distributed data centers, but are used to bypass the Internet as much as possible by peering directly with lower-tier ISPs. As a result, Google provides search results and e-mail access almost instantaneously, as if their data centers were running within one’s own computer.

- Many Internet commerce companies are now running their applications in the “cloud”—such as in Amazon’s EC2, in Microsoft’s Azure, or in the Alibaba Cloud.

1.8 Summary

Road-Mapping This Book

Our road map is the sequence of chapters of this book:

1. Computer Networks and the Internet
2. Application Layer
3. Transport Layer
4. Network Layer: Data Plane
5. Network Layer: Control Plane
6. The Link Layer and LANs
7. Wireless and Mobile Networks
8. Security in Computer Networks

Chapters 2 through 6 are the five core chapters of this book.

You should notice that these chapters are organized around the top four layers of the five-layer Internet protocol.

The second half of the book—Chapters 7 and 8—zooms in on two enormously important (and somewhat independent) topics in modern computer networking.

In Chapter 7, we examine wireless and mobile networks, including wireless LANs (including WiFi and Bluetooth), Cellular networks (including 4G and 5G), and mobility.

Chapter 8, which addresses security in computer networks, first looks at the underpinnings of encryption and network security, and then we examine how the basic theory is being applied in a broad range of Internet contexts.